



Αναφορά ΜΕΛΟΥΣ ΟΜΑΔΑΣ 1 Προγραμματιστικού Project

Στοιχεία μέλους ομάδας: {Καναβού Καλλιόπη, std170680@ac.eap.gr}

Τίτλος Project: Ανάπτυξη εφαρμογής για τη διαχείριση των ραντεβού μιας μικρής επιχείρησης

Τμήμα ΗΛΕ – 47

Ενότητα: ΠΛΗΠΡΟ

2025 – 2026

Ευχαριστίες	1
Καλή εξήγηση με τι ασχολήθηκε ο καθένας.....	1
Πόσες (κατά μέσο όρο εκτίμηση) ώρες εργάστηκε συνολικά αλλά και ανά εβδομάδα	1
Διαχείριση Ραντεβού	2
Ημερήσιο Πρόγραμμα.....	3
Αναζήτηση Ιστορικού	7
Λειτουργία Στατιστικά.....	7
Πρόβλημα.....	7
Επίλυση και τελική έκδοση	7
Καρτέλα ανά Ημέρα	7
Καρτέλα ανά Μήνα	7
Οι 10 Top Πελάτες	7
Απόδοση Υπαλλήλων	7
Παράρτημα Πηγαίου Κώδικα.....	7
Appointments.py	7
Stats.py	7
Πηγές	7

Ευχαριστίες

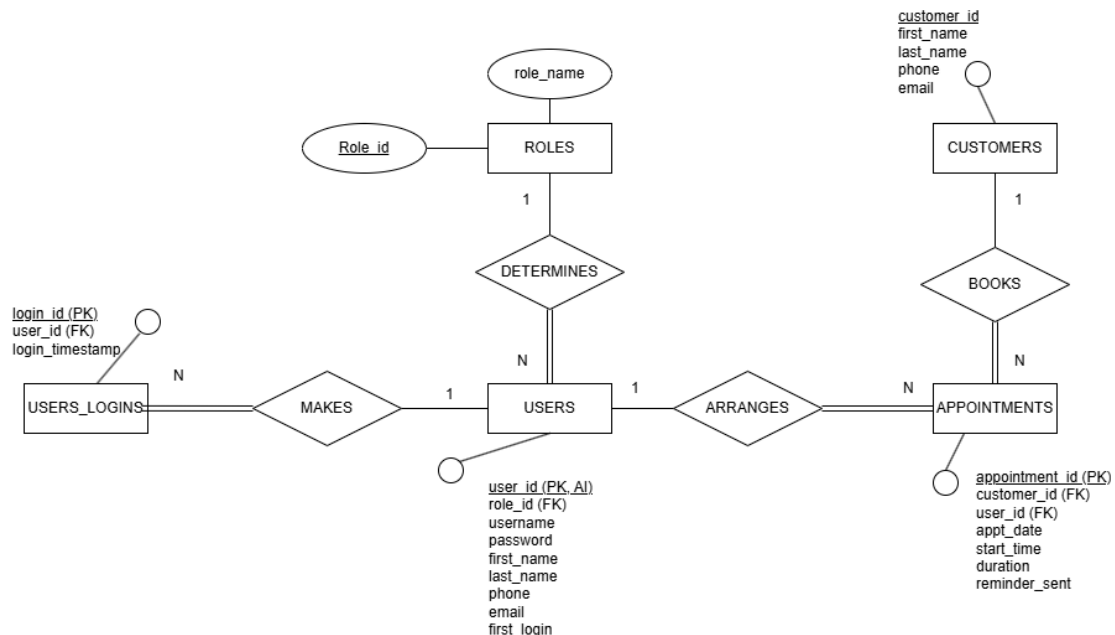
Θέλω να ευχαριστήσω ιδιαίτερα τους συναδέλφους της Ομάδας 1, διότι κατά τη διάρκεια της υλοποίησης του Project κάθε λεπτό και κάθε ώρα ήταν δίπλα μου πάντα.

Καλή εξήγηση με τι ασχολήθηκε ο καθένας

Ασχολήθηκα με την βάση δεδομένων στην αρχή. Κατόπιν συζήτησης με την ομάδα ανέλαβα να φτιάξω την βάση δεδομένων. Έπειτα από ομαδική συνάντηση χωρίσαμε τις αρμοδιότητες, και συνεπώς ασχολήθηκα με 1) την Διαχείριση των Ραντεβού και 2) με τα Στατιστικά στοιχεία. Φυσικά και η τεκμηρίωση στο ομαδικό Project είναι έναν κομμάτι που ασχολήθηκα κατόπιν συνεννόησης με όλα τα μέλη της ομάδας.

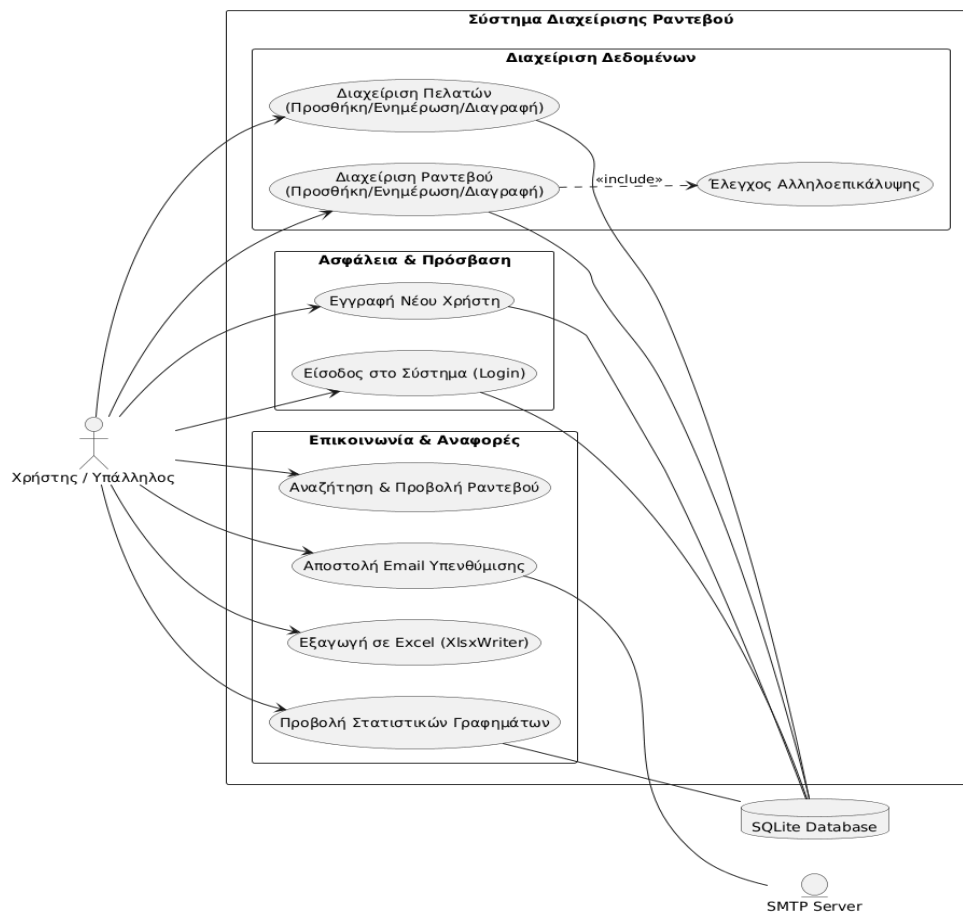
Πόσες (κατά μέσο όρο εκτίμηση) ώρες εργάστηκε συνολικά αλλά και ανά εβδομάδα

Στην αρχή που ξεκίνησε το project ασχολήθηκα αρκετά με τη βάση δεδομένων. Έφτιαξα το αρχικό ER και έπειτα σε SQLite υλοποίησα την βάση. Οι παραδοχές και το μοντέλο βρίσκονται στην ομαδική τεκμηρίωση. Το ER της βάσης είναι η παρακάτω εικόνα:



Εικόνα 1: ER Βάσης Δεδομένων της εφαρμογής

Ακολούθησε το μοντέλο πως περίπου θα στηθεί η εφαρμογή και πως περίπου θα κινηθούμε κατά τη διαδικασία της υλοποίησης με το Διάγραμμα UML Use Case - Μελέτη περίπτωσης:



Εικόνα 2: UML - Use Case

Διαχείριση Ραντεβού

Ημερήσιο Πρόγραμμα

Το κυρίως μέρος από το κομμάτι που ανέλαβα ήταν η Διαχείριση των ραντεβού. Αυτό πήρε αρκετό καιρό καθώς η Διαχείριση των Ραντεβού έχει αρκετές λειτουργίες που έπρεπε να υλοποιηθούν, όπως φαίνεται στην παρακάτω εικόνα.

RandeBoo Dashboard — admin

RandeBoo

Αρχική

Ραντεβού

Πελάτες

Αναζήτηση

Στατιστικά

Υπόμνημα

Χρήστες

Ρυθμίσεις

Αποσύνδεση

Ημερήσιο Πρόγραμμα

Αναζήτηση Ιστορικού

Διαχείριση Ραντεβού

Ημερομηνία: 02/06/2026

Προβολή Καταχώσεων

Σήμερα

Παύση

Διαγραφή

±/∓	Ώρα	Κατάσταση	Υπεύθυνος	Πελάτης	Σημειώσεις
+	09:00 - 09:30	ΕΛΕΓΧΕΡΟ			
-	09:30 - 10:00	1/1 Κρατήσεις			
—	—	ΡΑΝΤΕΒΟΥ	Πόπη Καναβού	Μονίκα Μεταλούτσι	
+	10:00 - 10:30	1/1 Κρατήσεις			
—	—	ΡΑΝΤΕΒΟΥ	Πόπη Καναβού	Αντώνη Παντζάρης	[ΝΕΟ] μόνιμος χρονοαπο
+	10:30 - 11:00	ΕΛΕΓΧΕΡΟ			
+	11:00 - 11:30	ΕΛΕΓΧΕΡΟ			
+	11:30 - 12:00	ΕΛΕΓΧΕΡΟ			
+	12:00 - 12:30	ΕΛΕΓΧΕΡΟ			
+	12:30 - 13:00	ΕΛΕΓΧΕΡΟ			
+	13:00 - 13:30	ΕΛΕΓΧΕΡΟ			
+	13:30 - 14:00	ΕΛΕΓΧΕΡΟ			
+	14:00 - 14:30	ΕΛΕΓΧΕΡΟ			
+	14:30 - 15:00	ΕΛΕΓΧΕΡΟ			
+	15:00 - 15:30	ΕΛΕΓΧΕΡΟ			
+	15:30 - 16:00	ΕΛΕΓΧΕΡΟ			
+	16:00 - 16:30	ΕΛΕΓΧΕΡΟ			
+	16:30 - 17:00	ΕΛΕΓΧΕΡΟ			
+	17:00 - 17:30	ΕΛΕΓΧΕΡΟ			
+	17:30 - 18:00	ΕΛΕΓΧΕΡΟ			
+	18:00 - 18:30	ΕΛΕΓΧΕΡΟ			
+	18:30 - 19:00	ΕΛΕΓΧΕΡΟ			
+	19:00 - 19:30	ΕΛΕΓΧΕΡΟ			
+	19:30 - 20:00	ΕΛΕΓΧΕΡΟ			
+	20:00 - 20:30	ΕΛΕΓΧΕΡΟ			
+	20:30 - 21:00	ΕΛΕΓΧΕΡΟ			

28°C

Ηλιοφάνεια

6:32 μμ

1/6/2026

Εικόνα 3: Διαχείριση Ραντεβού

Η παραπάνω εικόνα δείχνει ότι έχει δύο tab το Ημερήσιο Πρόγραμμα και την Αναζήτηση Ραντεβού.

Στο ημερήσιο πρόγραμμα ο χρήστης μπορεί να δημιουργήσει νέο, να τροποποιήσει ή να διαγράψει ένα ραντεβού. Τα ραντεβού κλείνονται επιλέγοντας μια συγκεκριμένη ημερομηνία από το ημερολόγιο. Σημειωτέον δε στις επίσημες αργίες της Ελλάδας και τις Κυριακές ο χρήστης ή ο πελάτης δεν μπορούν να κλείσουν κάποιο ραντεβού.

RandeBoo Dashboard — admin

RandeBoo

Αρχική
Ραντεβού
Πελάτες
Αναζήτηση
Στατιστικά
Υπόμνημα
Χρήστες
Ρυθμίσεις
Αποσύνδεση

Ημερήσιο Πρόγραμμα Αναζήτηση Ιστορικού

Διαχείριση Ραντεβού

Ημερομηνία: 01/06/2026 Προβολή Κλεισμένων Σήμερα Επιλέγουμε Διαγραφή

+/-	Όρα	Κατάσταση	Υπάλληλος	Πατέρας	Σημειώσεις
—	ΚΛΕΙΣΤΑ	Λόγω: Δευτέρα του Αγίου Πνεύματος			

Πληκτρολογήστε εδώ για αναζήτηση

28°C Ηλιοφάνεια 6:38 μμ 1/6/2026

RandeBoo Dashboard — admin

RandeBoo

Αρχική
Ραντεβού
Πελάτες
Αναζήτηση
Στατιστικά
Υπόμνημα
Χρήστες
Ρυθμίσεις
Αποσύνδεση

Ημερήσιο Πρόγραμμα Αναζήτηση Ιστορικού

Διαχείριση Ραντεβού

Ημερομηνία: 07/06/2026 Προβολή Κλεισμένων Σήμερα Επιλέγουμε Διαγραφή

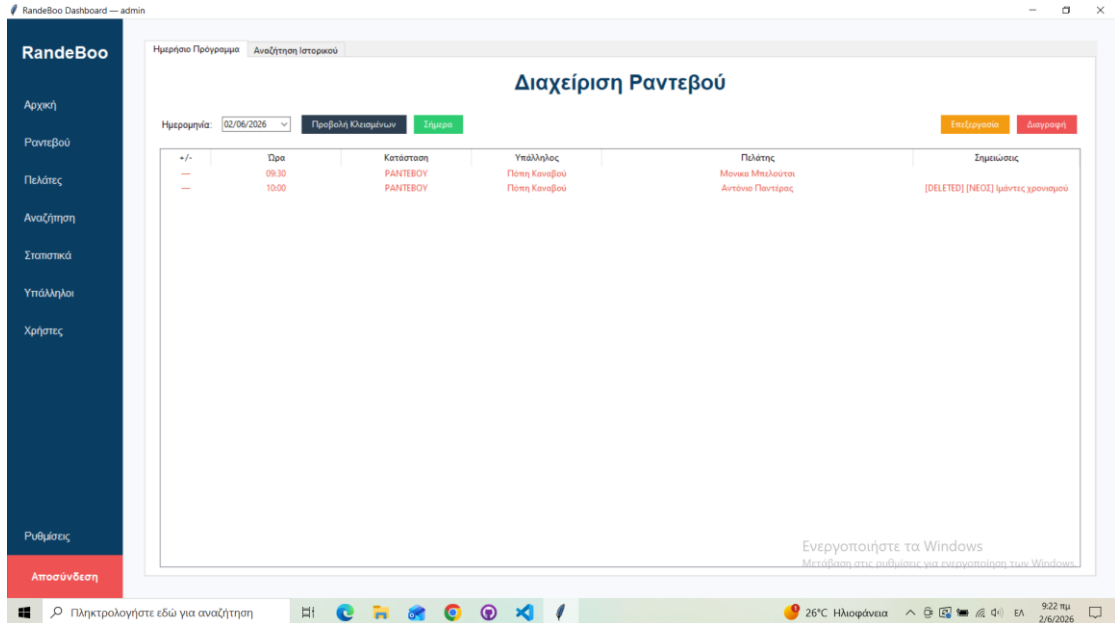
+/-	Όρα	Κατάσταση	Υπάλληλος	Πατέρας	Σημειώσεις
—	ΚΛΕΙΣΤΑ	Λόγω: Κυριακή			

Πληκτρολογήστε εδώ για αναζήτηση

28°C Ηλιοφάνεια 6:38 μμ 1/6/2026

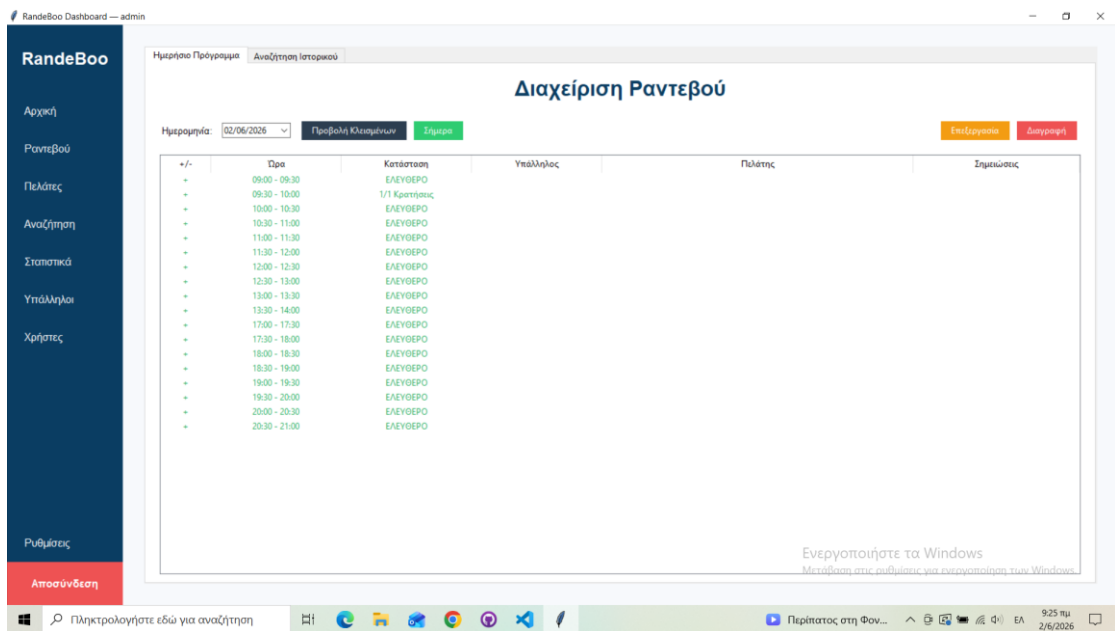
Εικόνες 4 & 5: Επίσημες Αργίες και Κυριακές

Μπορεί επίσης να επιλέξει προβολή κλεισμένων να εμφανιστούν όλα τα κλεισμένα ραντεβού



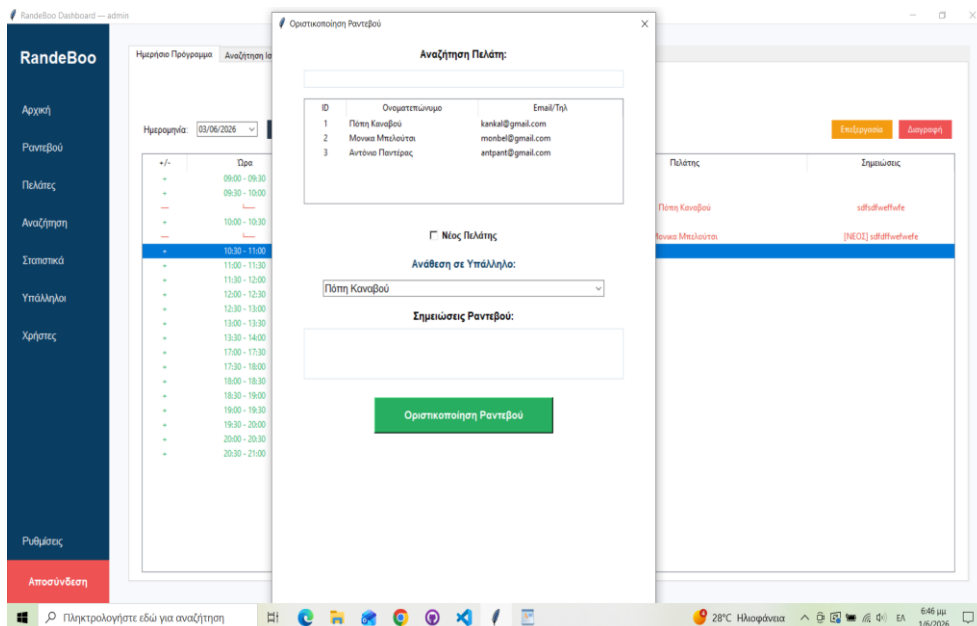
Εικόνα 6: Προβολή κλεισμένων

Μπορεί να δει τα ραντεβού της ημέρας από του κουμπί Σήμερα



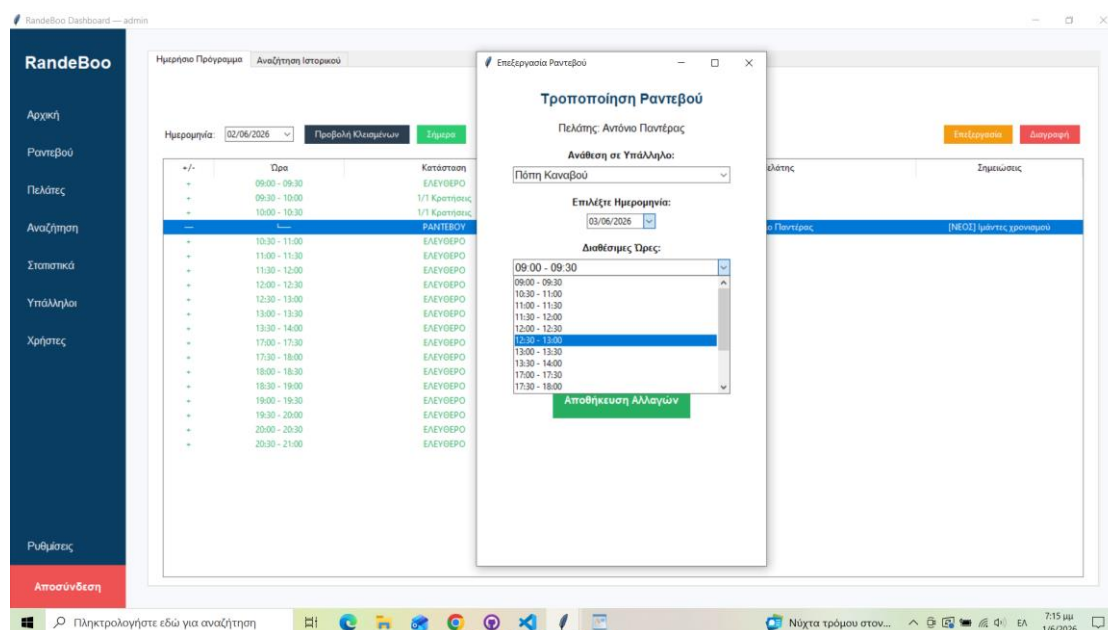
Εικόνα 7: Σήμερα

Για το κλείσιμο ενός ραντεβού έχοντας επιλέξει την ημερομηνία από το Ημερολόγιο, με διπλό κλικ πάνω στην ώρα ανοίγει η καρτέλα για το κλείσιμο ενός νέου ραντεβού. Ο χρήστης μπορεί είτε να επιλέξει έναν πελάτη είτε να προσθέσει έναν καινούργιο πελάτη. Για το κλείσιμο του ραντεβού ο χρήστης επιλέγει Οριστικοποίηση Ραντεβού.



Εικόνα 8: Κλείσιμο νέου ραντεβού

Τα ραντεβού τροποποιούνται από το πορτοκαλί κουμπί ΕΠΕΞΕΡΓΑΣΙΑ. Ο χρήστης μπορεί να επιλέξει άλλη μέρα και ώρα. Η λίστα με τις ώρες εμφανίζει μόνο τα διαθέσιμα ραντεβού που δεν είναι κλεισμένα από άλλον πελάτη.



Εικόνα 9: Drop-Down menu διαθέσιμων ραντεβού

Επιπλέον η διαγραφή ενός ραντεβού γίνεται από το κουμπί ΔΙΑΓΡΑΦΗ. Εμφανίζεται μήνυμα επιβεβαίωσης αν ο χρήστης θέλει να διαγράψει το ραντεβού. Αν ο χρήστης επιλέξει ΝΑΙ το ραντεβού διαγράφεται και η ώρα εμφανίζεται διαθέσιμη και με πράσινο χρώμα. Στο όχι δεν αποθηκεύονται τα ραντεβού.

RandeBoo Dashboard — admin

RandeBoo

- Αρχική
- Ραντεβού
- Πελάτες
- Αναζήτηση
- Στατιστικά
- Υπάλληλοι
- Χρήστες
- Ρυθμίσεις
- Αποσύνδεση

Ημερήσιο Πρόγραμμα Αναζήτηση Ιστορικού

Διαχείριση Ραντεβού

Ημερομηνία: 02/06/2026 Προβολή Κλεισμένων Σήμερα

Επιλέγουμε Διαγραφή

+/-	Ωρα	Κατάσταση	Υπάλληλος	Πελάτης	Σημειώσεις
+	09:00 - 09:30	ΕΛΕΥΘΕΡΟ			
+	09:30 - 10:00	1/1 Κρατήσεις			
+	10:00 - 10:30	1/1 Κρατήσεις			
—	—	ΡΑΝΤΕΒΟΥ	Πόπη Κονοβού	Αντόνιο Παντίρας	[ΝΕΟ] (μόνιμος χρόνος)
+	10:30 - 11:00	ΕΛΕΥΘΕΡΟ			
+	11:00 - 11:30	ΕΛΕΥΘΕΡΟ			
+	11:30 - 12:00	ΕΛΕΥΘΕΡΟ			
+	12:00 - 12:30	ΕΛΕΥΘΕΡΟ			
+	12:30 - 13:00	ΕΛΕΥΘΕΡΟ			
+	13:00 - 13:30	ΕΛΕΥΘΕΡΟ			
+	13:30 - 14:00	ΕΛΕΥΘΕΡΟ			
+	17:00 - 17:30	ΕΛΕΥΘΕΡΟ			
+	17:30 - 18:00	ΕΛΕΥΘΕΡΟ			
+	18:00 - 18:30	ΕΛΕΥΘΕΡΟ			
+	18:30 - 19:00	ΕΛΕΥΘΕΡΟ			
+	19:00 - 19:30	ΕΛΕΥΘΕΡΟ			
+	19:30 - 20:00	ΕΛΕΥΘΕΡΟ			
+	20:00 - 20:30	ΕΛΕΥΘΕΡΟ			
+	20:30 - 21:00	ΕΛΕΥΘΕΡΟ			

Επιβεβαίωση

Ακύρωση ραντεβού για: Αντόνιο Παντίρας

Ναι Όχι

Πληκτρολογήστε εδώ για αναζήτηση

27°C Ηλιοφάνεια 7:32 μμ 1/6/2026

RandeBoo Dashboard — admin

RandeBoo

- Αρχική
- Ραντεβού
- Πελάτες
- Αναζήτηση
- Στατιστικά
- Υπάλληλοι
- Χρήστες
- Ρυθμίσεις
- Αποσύνδεση

Ημερήσιο Πρόγραμμα Αναζήτηση Ιστορικού

Διαχείριση Ραντεβού

Ημερομηνία: 02/06/2026 Προβολή Κλεισμένων Σήμερα

Επιλέγουμε Διαγραφή

+/-	Ωρα	Κατάσταση	Υπάλληλος	Πελάτης	Σημειώσεις
+	09:00 - 09:30	ΕΛΕΥΘΕΡΟ			
+	09:30 - 10:00	1/1 Κρατήσεις			
+	10:00 - 10:30	1/1 Κρατήσεις			
—	—	ΡΑΝΤΕΒΟΥ	Πόπη Κονοβού	Αντόνιο Παντίρας	[ΝΕΟ] (μόνιμος χρόνος)
+	10:30 - 11:00	ΕΛΕΥΘΕΡΟ			
+	11:00 - 11:30	ΕΛΕΥΘΕΡΟ			
+	11:30 - 12:00	ΕΛΕΥΘΕΡΟ			
+	12:00 - 12:30	ΕΛΕΥΘΕΡΟ			
+	12:30 - 13:00	ΕΛΕΥΘΕΡΟ			
+	13:00 - 13:30	ΕΛΕΥΘΕΡΟ			
+	13:30 - 14:00	ΕΛΕΥΘΕΡΟ			
+	17:00 - 17:30	ΕΛΕΥΘΕΡΟ			
+	17:30 - 18:00	ΕΛΕΥΘΕΡΟ			
+	18:00 - 18:30	ΕΛΕΥΘΕΡΟ			
+	18:30 - 19:00	ΕΛΕΥΘΕΡΟ			
+	19:00 - 19:30	ΕΛΕΥΘΕΡΟ			
+	19:30 - 20:00	ΕΛΕΥΘΕΡΟ			
+	20:00 - 20:30	ΕΛΕΥΘΕΡΟ			
+	20:30 - 21:00	ΕΛΕΥΘΕΡΟ			

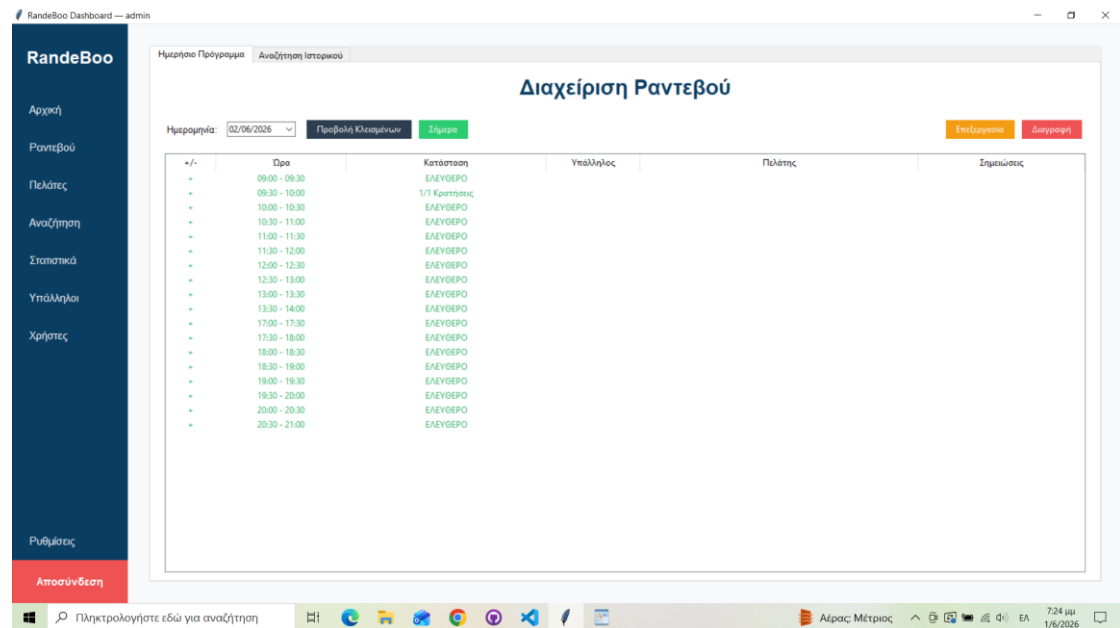
Πληροφορία

Το ραντεβού ακυρώθηκε επιτυχώς στο ιστορικό.

ΟΚ

Πληκτρολογήστε εδώ για αναζήτηση

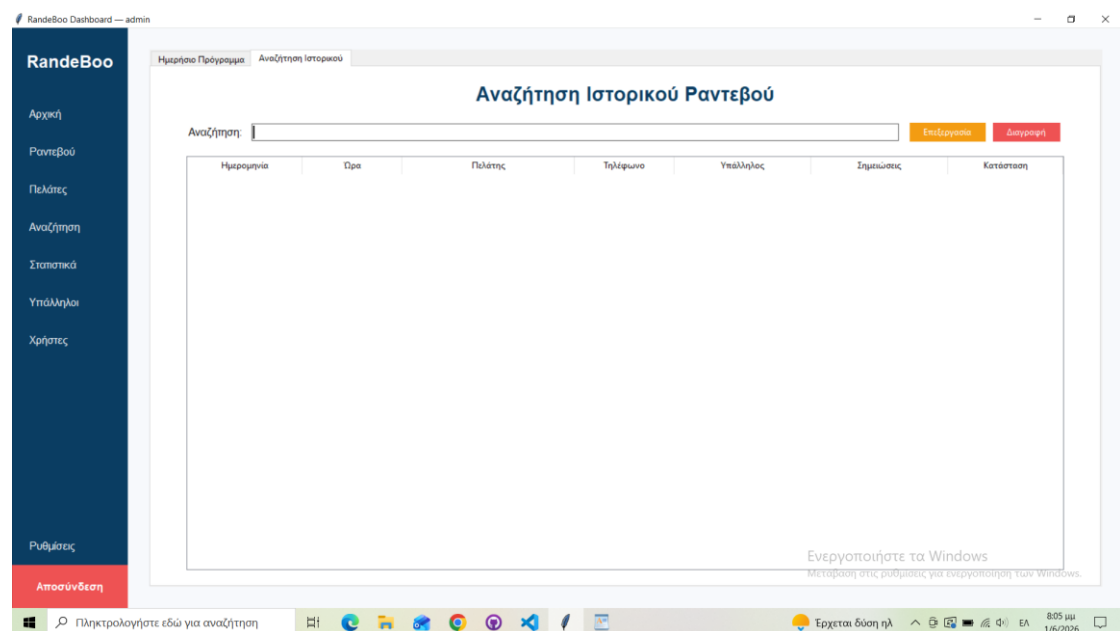
27°C Ηλιοφάνεια 7:33 μμ 1/6/2026



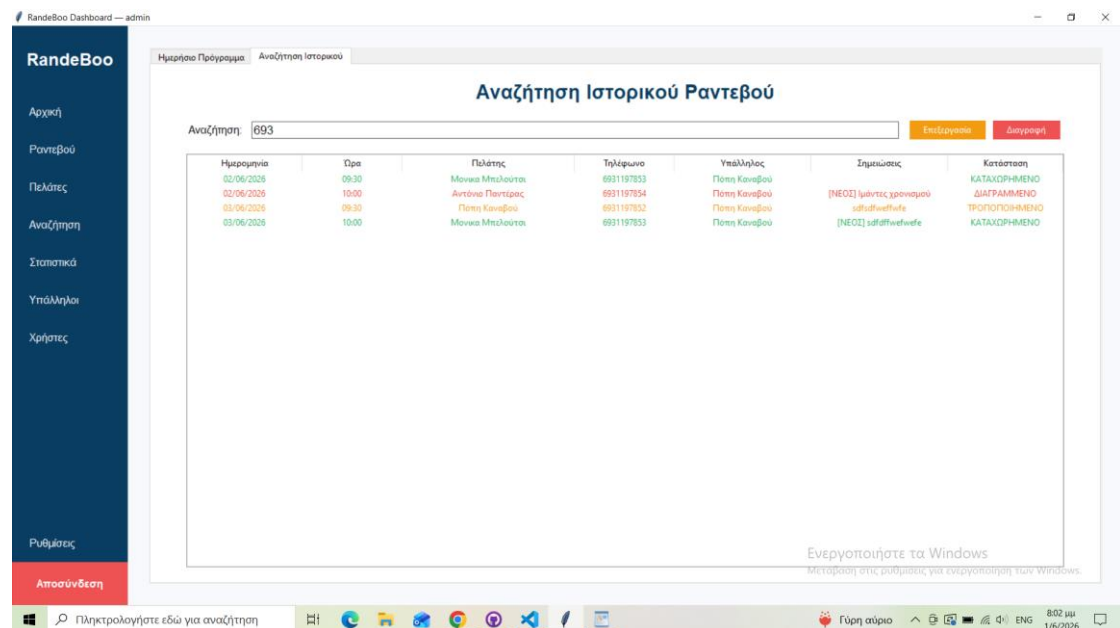
Εικόνες 10, 11 & 12: Διαγραφή Ραντεβού

Αναζήτηση Ιστορικού

Προέκυψε η ανάγκη κατά τη διάρκεια της υλοποίησης και πριν την ολοκλήρωση της λειτουργίας της αναζήτησης το ερώτημα του πως μπορεί κάποιος να δει τα διαθέσιμα ραντεβού του. Έτσι δημιουργήθηκε ένα δεύτερο tab με τίτλο Αναζήτηση Ραντεβού. Έτσι ο χρήστης μπορεί να ψάξει είτε με το όνομα είτε με το τηλέφωνο είτε με το e-mail τα ραντεβού του.



Εικόνα 13: Ημερήσιο Πρόγραμμα



Εικόνα 14: Αναζήτηση με το κινητό

Επιπλέον και σε αυτό το σημείο ο χρήστης μπορεί είτε να τροποποιήσει είτε να διαγράψει ένα ραντεβού. Τα επεξεργασμένα/ τροποποιημένα ραντεβού εμφανίζονται με πορτοκαλί χρώμα και τα διαγραμμένα με κόκκινο χρώμα.

Λειτουργία Στατιστικά

Στην λειτουργία Στατιστικά με βοήθησε πολύ ο συνάδελφος Χρήστος, διότι αντιμετωπίσαμε τεχνικά ζητήματα.

Πρόβλημα

Μετά την επιτυχή εισαγωγή μεγάλου όγκου δεδομένων (π.χ. δοκιμαστικών ραντεβού που εκτείνονταν σε βάθος μηνών ή ετών), τα γραφήματα παρουσίασαν σοβαρά θέματα εμφάνισης (UI Overcrowding):

- Συνωστισμός στον Άξονα Χ: Το γράφημα «Ανά Ημέρα» προσπαθούσε να σχεδιάσει κάθε ημερομηνία της βάσης ξεχωριστά. Λόγω του περιορισμένου χώρου, εκατοντάδες ετικέτες ημερομηνιών σχεδιάζονταν η μία πάνω στην άλλη, δημιουργώντας μια αδιάβαστη μαύρη μάζα στο κάτω μέρος του γραφήματος.
- Μη-Διαδραστική Πληροφορία: Τα γραφήματα εμφάνιζαν στατικά όλα τα ιστορικά δεδομένα της βάσης, κάνοντας αδύνατη την ανάλυση συγκεκριμένων περιόδων (π.χ. τι κίνηση υπήρξε σήμερα ή αυτόν τον μήνα).
- Έλλειψη Συνοχής: Ο τίτλος του γραφήματος υποσχόταν εμφάνιση των «Τελευταίων 30 ημερών», αλλά το SQL query επέστρεφε όλα τα δεδομένα της βάσης χωρίς χρονικό φιλτράρισμα.

Έπρεπε να Αλλαχθεί το δυναμικό φιλτράρισμα όπου ο χρήστης έπρεπε να μπορεί να επιλέξει ποια χρονική περίοδο θέλει να μελετήσει (συγκεκριμένη ημέρα ή συγκεκριμένο μήνα).

Για τον Άξονα Χ, την ημερήσια προβολή, οι ετικέτες των ωρών έπρεπε να αραιωθούν (π.χ. εμφάνιση ανά 1 ώρα αντί για 30 λεπτά). Για τη μηνιαία προβολή, οι ετικέτες των ημερών έπρεπε να εμφανίζονται ανά 5 ημέρες (π.χ. 1, 6, 11, 16...).

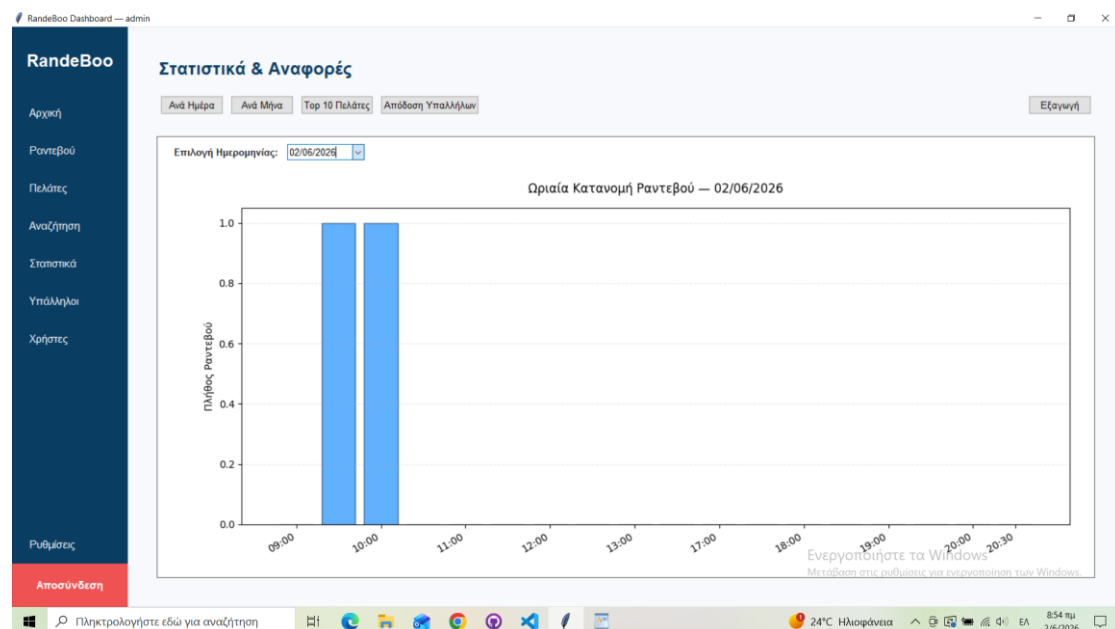
Σύνδεση με τις Ρυθμίσεις Οι ώρες και τα slots της ημερήσιας κατανομής έπρεπε να διαβάζονται δυναμικά από το ωράριο λειτουργίας της επιχείρησης στη βάση δεδομένων (weekly_schedule), ώστε η εφαρμογή να προσαρμόζεται αυτόματα σε κάθε εταιρεία.

Τέλος το Το μηνιαίο γράφημα έπρεπε να μετατραπεί σε ένα ομαλό κύμα με απαλό ημιδιαφανές γέμισμα (Area Chart), προσδίδοντας μια premium αίσθηση στο Aegean Theme.

Επίλυση και τελική έκδοση

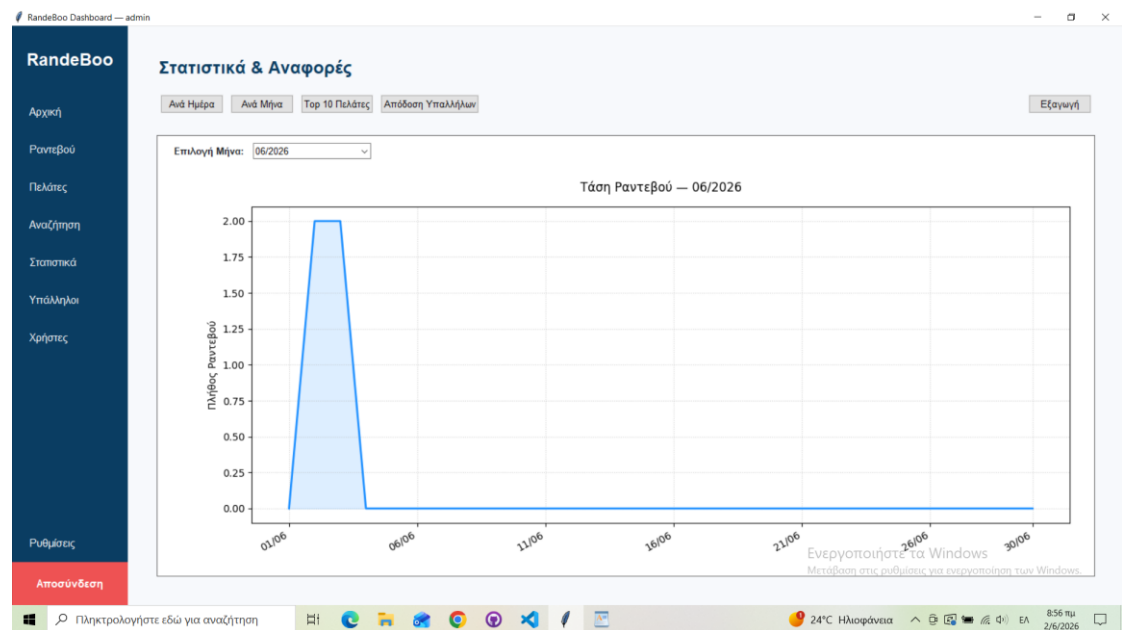
Καρτέλα ανά Ημέρα

Στην καρτέλα «Ανά Ημέρα» ο χρήστης επιλέγει μια ημερομηνία από το ημερολόγιο και εμφανίζεται το γράφημα ανα ημέρα.



Καρτέλα ανά Μήνα

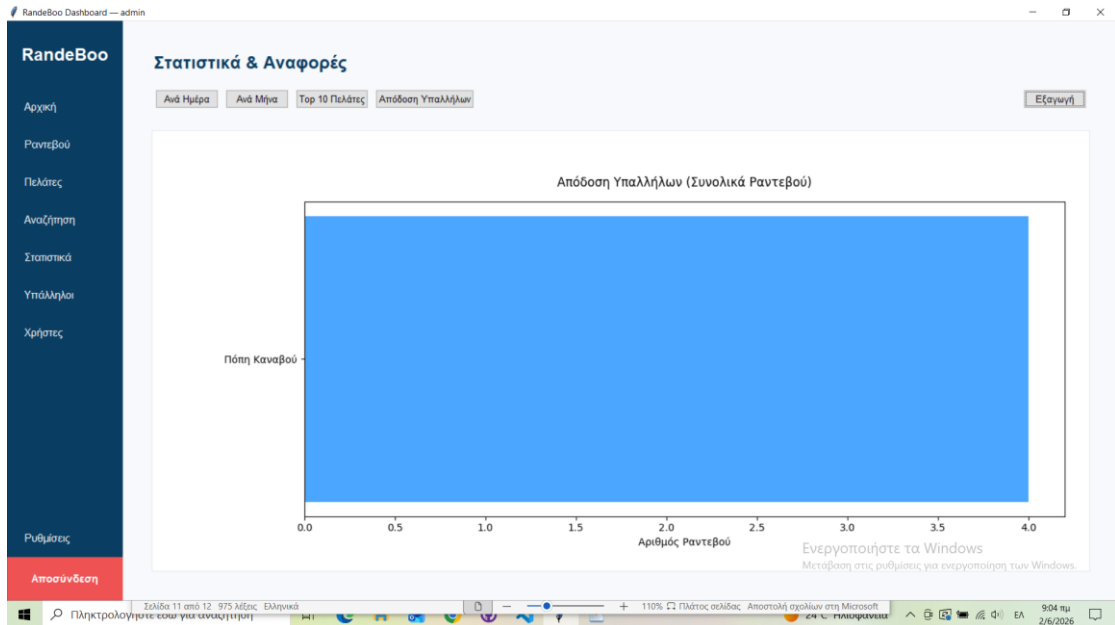
Ο χρήστης επιλέγει το κουμπί Ανά Μήνα και εμφανίζεται η μηνιαία κατανομή των ραντεβού



Οι 10 Top Πελάτες



Απόδοση Υπαλλήλων



Παράρτημα Πηγαίου Κώδικα

Appointments.py

Ο πηγαίος κώδικας του appointments.py είναι ο παρακάτω

```
import tkinter as tk
from tkinter import messagebox, ttk
import json
import datetime
import holidays
from tkcalendar import DateEntry

import database

def is_holiday(date_obj):
    """Ελέγχου αν η ημερομηνία είναι επίσημη αργία ή Κυριακή."""
    # 1. Check Greece Holidays
    gr_holidays = holidays.Greece(years=date_obj.year, language='el')
    if date_obj in gr_holidays:
        return True, gr_holidays.get(date_obj)

    # 2. Check Business Blackout Dates
    settings = database.get_business_settings()
    blackouts = settings.get("blackout_dates", "").split(",")
    iso_date = date_obj.isoformat()
    if iso_date in blackouts:
        return True, "Αργία Επιχείρησης"
```

```

        return False, None

class CustomerSelector(tk.Toplevel):
    def __init__(self, parent, callback):
        super().__init__(parent)
        self.title("Οριστικοποίηση Παντεβού")
        self.geometry("600x820")
        self.configure(bg="white")
        self.callback = callback
        self.transient(parent)
        self.grab_set()

        # 1. ΠΕΡΙΕΚΤΗΣ ΠΕΛΑΤΗ (Εναλλαγή Αναζήτησης / Νέας Φόρμας)
        self.customer_container = tk.Frame(self, bg="white")
        self.customer_container.pack(fill="x", pady=10)

        # --- A. Frame Αναζήτησης (Existing Mode) ---
        self.frame_search = tk.Frame(self.customer_container,
        bg="white")
        self.frame_search.pack(fill="x") # Αρχικά ορατό

        tk.Label(self.frame_search, text="Αναζήτηση Πελάτη:",
        font=("Arial", 11, "bold"), bg="white").pack(pady=5)

        self.search_entry = tk.Entry(self.frame_search, font=("Arial",
        12), relief="flat",
        highlightthickness=1,
        highlightbackground="#D9E2EC")
        self.search_entry.pack(pady=5, padx=50, fill="x")
        self.search_entry.bind("<KeyRelease>", self.update_list)

        self.tree = ttk.Treeview(self.frame_search, columns=("ID",
        "Name", "Info"), show="headings", height=6)
        self.tree.heading("ID", text="ID"); self.tree.heading("Name",
        text="Ονοματεπώνυμο"); self.tree.heading("Info", text="Email/Τηλ")
        self.tree.column("ID", width=40, anchor="center");
        self.tree.column("Name", width=180); self.tree.column("Info",
        width=200)
        self.tree.pack(fill="x", padx=50, pady=10)

        # Binding για τη ροδέλα του ποντικιού στον πίνακα
        self.tree.bind("<MouseWheel>", self._on_tree_mousewheel)

        # --- B. Frame Νέου Πελάτη (New Mode) ---

```

```

        self.frame_new = tk.LabelFrame(self.customer_container,
text="Στοιχεία Νέου Πελάτη",
                                bg="white", font=("Arial", 10,
"bold"), padx=20, pady=15)

        self.entries_new = {}
        for label_text, key in [("Όνομα *:", "fname"), ("Επώνυμο *:",
"lname"), ("Τηλέφωνο:", "phone"), ("Email *:", "email")]:
            tk.Label(self.frame_new, text=label_text, bg="white",
font=("Arial", 9)).pack(anchor="w")
            ent = tk.Entry(self.frame_new, font=("Arial", 11),
relief="flat", highlightthickness=1, highlightbackground="#D9E2EC")
            ent.pack(fill="x", pady=(2, 8))
            self.entries_new[key] = ent

        # 2. TO CHECKBOX (Εναλλαγή Λειτουργίας)
        self.is_new_customer = tk.BooleanVar(value=False)
        self.chk_new = tk.Checkbutton(self, text="Νέος Πελάτης",
                                variable=self.is_new_customer,
bg="white",
                                font=("Arial", 10, "bold"),
command=self.toggle_mode)
        self.chk_new.pack(pady=10)

        # 3. ΛΟΙΠΑ ΣΤΟΙΧΕΙΑ ΠΑΝΤΕΒΟΥ
        tk.Label(self, text="Ανάθεση σε Υπάλληλο:", font=("Arial", 11,
"bold"), bg="white", fg="#0A3D62").pack(pady=5)
        self.all_employees = database.get_active_employees()
        emp_names = [f"{e['first_name']} {e['last_name']}" for e in
self.all_employees]
        self.emp_combo = ttk.Combobox(self, values=emp_names,
state="readonly", font=("Arial", 11))
        self.emp_combo.pack(pady=5, padx=80, fill="x")
        if emp_names: self.emp_combo.current(0)

        tk.Label(self, text="Σημειώσεις Παντεβού:", font=("Arial", 11,
"bold"), bg="white").pack(pady=(10, 0))
        self.notes_text = tk.Text(self, height=4, font=("Arial", 10),
relief="flat", highlightthickness=1, highlightbackground="#D9E2EC")
        self.notes_text.pack(pady=5, padx=50, fill="x")

        tk.Button(self, text="Οριστικοποίηση Παντεβού",
command=self.select_and_close,
                                bg="#27ae60", fg="white", font=("Arial", 11,
"bold"), pady=10, width=30).pack(pady=20)

```

```

# Αρχική φόρτωση δεδομένων
self.load_all_data()

def toggle_mode(self):
    """Εναλλαγή μεταξύ αναζήτησης και φόρμας νέου πελάτη."""
    if self.is_new_customer.get():
        self.frame_search.pack_forget()
        self.frame_new.pack(fill="x", padx=50)
    else:
        self.frame_new.pack_forget()
        self.frame_search.pack(fill="x")

def _on_tree_mousewheel(self, event):
    """Λειτουργία ροδέλας ποντικιού στον πίνακα."""
    if self.tree.wininfo_exists():
        self.tree.yview_scroll(int(-1 * (event.delta / 120)),
"units")

def load_all_data(self):
    """Φόρτωση όλων των πελατών."""
    for item in self.tree.get_children(): self.tree.delete(item)
    results = database.get_all_customers()
    for c in results:
        c_id = c.get('customer_id') or c.get('id')
        name = f"{c.get('first_name', '')} {c.get('last_name', '')}"
        self.tree.insert("", "end", values=(c_id, name,
c.get('email', '-')))

def update_list(self, event=None):
    """Δυναμική αναζήτηση."""
    query = self.search_entry.get().strip().lower()

    # Αν το query είναι μικρό, φόρτωσε όλα τα δεδομένα
    if len(query) < 2:
        return self.load_all_data()

    # Καθαρισμός πίνακα
    for item in self.tree.get_children():
        self.tree.delete(item)

    customers = database.get_all_customers()

    for c in customers:
        name = f"{c.get('first_name', '')} {c.get('last_name', '')}"
        email = str(c.get('email', '')).lower()
        phone = str(c.get('phone', '-'))

```



```

        # Καθαρισμός για την αναζήτηση (χωρίς κενά)
        clean_phone = phone.replace(" ", "")
        clean_query = query.replace(" ", "")

        # 1. Ενημέρωση Φίλτρου: Έλεγχος σε Όνομα, Email και
Τηλέφωνο
        if (query in name.lower() or
            query in email or
            clean_query in clean_phone):

            # 2. Ενημέρωση Εμφάνισης: Email και Τηλέφωνο μαζί στη
στήλη Info
            contact_info = f"{email} | {phone}" if email and email
!= "-" else phone

            self.tree.insert("", "end", values=(
                c.get('customer_id') or c.get('id'),
                name,
                contact_info
            ))

    def select_and_close(self):
        """Οριστικοποίηση με αυστηρό έλεγχο μοναδικότητας και διακοπή
σε σφάλμα."""
        notes = self.notes_text.get("1.0", "end-1c").strip()
        cust_id = None

        if self.is_new_customer.get():
            fn = self.entries_new['fname'].get().strip()
            ln = self.entries_new['lname'].get().strip()
            em = self.entries_new['email'].get().strip()
            ph = self.entries_new['phone'].get().strip()

            if not fn or not ln or not em or not ph:
                return messagebox.showwarning("Προσοχή", "Παρακαλώ
συμπληρώστε Όνομα, Επώνυμο, Τηλέφωνο και Email.")

            if len(ph)<10:
                return messagebox.showwarning("Προσοχή", "Παρακαλώ
εισάγετε έγκυρο τηλέφωνο.")

            if "@" not in em or "." not in em:
                messagebox.showwarning("Σφάλμα", "Το e-mail δεν
φαίνεται έγκυρο!")

            return

```

```

        # 1. ΕΛΕΓΧΟΣ EMAIL: Αν υπάρχει, σταματάμε (return)
        try:
            existing_email = database.get_customer_by_email(em)
            if existing_email:
                return messagebox.showerror("Σφάλμα", f"To Email '{em}' χρησιμοποιείται ήδη!")
            except: pass

        # 2. ΔΗΜΙΟΥΡΓΙΑ ΠΕΛΑΤΗ: Αν αποτύχει (π.χ. UNIQUE τηλέφωνο), σταματάμε (return)
        try:
            cust_id = database.create_customer(fn, ln, ph or None, em)

            notes = f"[ΝΕΟΣ] {notes}".strip()
        except Exception:
            # Εδώ σταματάμε την εκτέλεση για να μην προσπαθήσει να κλείσει ραντεβού
            return messagebox.showerror("Σφάλμα", f"To Τηλέφωνο '{ph}' χρησιμοποιείται ήδη!")
        else:
            # Λογική για υπάρχοντα πελάτη
            sel = self.tree.selection()
            if not sel:
                return messagebox.showwarning("Προσοχή", "Παρακαλώ επιλέξτε έναν πελάτη από τη λίστα.")

            # Παίρνουμε το ID (1η τιμή από τα values)
            item_data = self.tree.item(sel[0])
            vals = item_data.get("values")
            if vals:
                cust_id = vals[0] # Το ID είναι πάντα η πρώτη στήλη

        # 3. Λήψη ID Υπαλλήλου
        idx = self.emp_combo.current()
        emp_id = self.all_employees[idx]['employee_id'] if idx != -1 else None

        # 4. Επιστροφή στο κεντρικό παράθυρο MONO αν έχουμε έγκυρο cust_id
        if cust_id:
            # Σημαντικό: Το callback θα καλέσει την confirm_booking
            self.callback(cust_id, emp_id, notes)
            self.destroy()
        else:
            messagebox.showerror("Σφάλμα", "Δεν βρέθηκε ID πελάτη. Δοκιμάστε ξανά.")

```

```

class EditAppointmentWindow(tk.Toplevel):
    def __init__(self, parent, appt, on_save_callback):
        super().__init__(parent)
        self.title("Επεξεργασία Ραντεβού")
        self.geometry("400x680")
        self.configure(bg="white")
        self.parent = parent
        self._appt = appt
        self.on_save_callback = on_save_callback

        tk.Label(self, text="Τροποποίηση Ραντεβού", font=("Arial", 14,
"bold"), bg="white", fg="#0A3D62").pack(pady=(20, 10))

        # Πληροφορίες Πελάτη
        tk.Label(self, text=f"Πελάτης: {appt.get('customer_name',
'Άγνωστος')}", font=("Arial", 11), bg="white").pack(pady=5)

        # ΕΠΙΛΟΓΗ ΥΠΑΛΛΗΛΟΥ
        tk.Label(self, text="Ανάθεση σε Υπάλληλο:", bg="white",
font=("Arial", 10, "bold")).pack(pady=(10, 0))
        self.all_employees = database.get_active_employees()
        emp_names = [f"{e['first_name']} {e['last_name']}" for e in
self.all_employees]
        self.emp_combo = ttk.Combobox(self, values=emp_names,
font=("Arial", 11), state="readonly")
        self.emp_combo.pack(pady=5, padx=50, fill="x")

        current_emp = appt.get('employee_name')
        if current_emp in emp_names:
            self.emp_combo.set(current_emp)
        elif emp_names:
            self.emp_combo.current(0)

        # Ημερομηνία
        tk.Label(self, text="Επιλέξτε Ημερομηνία:", bg="white",
font=("Arial", 10, "bold")).pack(pady=(10, 0))
        self.new_cal = DateEntry(self, width=12,
date_pattern="dd/mm/yyyy", background="#0A3D62", foreground="white")
        if appt.get('appt_date'):
            try:
                d_parts = appt['appt_date'].split('-')
                self.new_cal.set_date(datetime.date(int(d_parts[0]),
int(d_parts[1]), int(d_parts[2])))
            except: pass

        self.new_cal.pack(pady=5)

```

```

        self.new_cal.bind("<<DateEntrySelected>>", lambda e:
self.update_available_hours())

        # Ώρα
        tk.Label(self, text="Διαθέσιμες Ώρες:", bg="white",
font=("Arial", 10, "bold")).pack(pady=(10, 0))
        self.time_combo = ttk.Combobox(self, font=("Arial", 11),
state="readonly")
        self.time_combo.pack(pady=5, padx=50, fill="x")

        # Σημειώσεις
        tk.Label(self, text="Σημειώσεις Παντεβού:", bg="white",
font=("Arial", 10, "bold")).pack(pady=(10, 0))
        self.notes_text = tk.Text(self, height=4, font=("Arial", 10),
relief="flat", highlightthickness=1, highlightbackground="#D9E2EC")
        self.notes_text.pack(pady=5, padx=50, fill="x")

        # Προ-συμπλήρωση σημειώσεων (χωρίς τα εσωτερικά tags για
καθαρή επεξεργασία)
        raw_notes = appt.get('notes', "")
        display_notes = str(raw_notes).replace("[MODIFIED]",
"").replace("[DELETED]", "").strip()
        self.notes_text.insert("1.0", display_notes)

        # Κουμπί Αποθήκευσης
        tk.Button(
            self, text="Αποθήκευση Αλλαγών",
            command=self.save, bg="#27ae60", fg="white",
            font=("Arial", 11, "bold"), pady=10, width=20,
relief="flat", cursor="hand2"
        ).pack(pady=30)

        # Αρχική φόρτωση ωρών
        self.update_available_hours()
        self.lift()
        self.focus_force()

    def save(self):
        """Οριστική διόρθωση σειράς δεδομένων και ελέγχου ορίου."""
        full_slot = self.time_combo.get()
        if not full_slot or "Πλήρες" in full_slot: return

        new_time = full_slot.split(" - ")[0].strip()[:5]
        new_date = self.new_cal.get() # dd/mm/yyyy

```

```

# --- ΕΛΕΓΧΟΣ ΟΡΙΟΥ ΒΑΣΕΙ ΕΝΕΡΓΩΝ ΥΠΑΛΛΗΛΩΝ ΠΡΙΝ ΤΗΝ
ΑΠΟΘΗΚΕΥΣΗ ---
    try:
        d_parts = new_date.split('/')
        db_date = f"{d_parts[2]}-{d_parts[1]}-{d_parts[0]}"
        appts = database.get_day_appointments(db_date)

        # Δυναμικό όριο βάσει ενεργών υπαλλήλων
        active_emp_count = len(database.get_active_employees())
        limit = max(1, active_emp_count)

        count = sum(1 for a in appts if str(a['start_time'][:5]
== new_time
                                and a['appointment_id'] !=
self._appt['appointment_id']
                                and "[DELETED]" not in str(a.get('notes',
"")))

        if count >= limit:
            messagebox.showerror("Πλήρες Slot", f"To slot
{new_time} έχει ήδη {limit} ραντεβού!")
            return
        except: pass

        selected_idx = self.emp_combo.current()
        new_emp_id = self.all_employees[selected_idx]['employee_id']
        if selected_idx != -1 else None

        # Λήψη των νέων σημειώσεων από το widget
        new_notes = self.notes_text.get("1.0", "end-1c").strip()

        self.on_save_callback(
            self._appt['appointment_id'], # 1. id
            self._appt['customer_id'],    # 2. customer_id
            self._appt.get('user_id', 1), # 3. user_id
            new_date,                      # 4. date
            new_time,                      # 5. time
            30,                           # 6. duration
            new_notes,                     # 7. notes
            new_emp_id                     # 8. employee_id
        )
        self.destroy()

    def update_available_hours(self):
        """Υπολογίζει τις διαθέσιμες ώρες και αφαιρεί τα γεμάτα
slots."""
        sel_date_obj = self.new_cal.get_date()
        date_for_db = sel_date_obj.strftime('%Y-%m-%d')

```

```

        slots = self.parent.generate_dynamic_slots(sel_date_obj)

        try:
            appts = database.get_day_appointments(date_for_db)
            booked_times = [str(a['start_time'])[:5] for a in appts if
"[DELETED]" not in str(a.get('notes', ""))]

            # Δυναμικό όριο βάσει ενεργών υπαλλήλων
            active_emp_count = len(database.get_active_employees())
            limit = max(1, active_emp_count)

            current_appt_time = str(self._appt.get('appt_time',
''))[:5]
            current_appt_date = self._appt.get('appt_date', '')

            available = []
            for s in slots:
                t = s.split(" - ")[0]
                count = booked_times.count(t)

                if count < limit or (t == current_appt_time and
date_for_db == current_appt_date):
                    available.append(s)

            self.time_combo['values'] = available

            if available:
                found_current = False
                for idx, val in enumerate(available):
                    if val.startswith(current_appt_time):
                        self.time_combo.current(idx)
                        found_current = True
                        break
                if not found_current:
                    self.time_combo.current(0)
            else:
                self.time_combo.set("Πλήρες Πρόγραμμα")

        except Exception as e:
            print(f"Σφάλμα στο φιλτράρισμα: {e}")
            self.time_combo['values'] = slots

class CalendarView(tk.Frame):
    def __init__(self, parent: tk.Widget, user: dict = None) -> None:
        super().__init__(parent, bg="white")

```

```

        self.user = user or {"id": 1, "role_id": 1} # Fallback for
standalone run
        self.role_id = self.user.get("role_id", 2)

        tk.Label(
            self, text="Διαχείριση Παντεβού", font=("Arial", 22,
"bold"),
            bg="white", fg="#0A3D62"
        ).pack(pady=15)

        # Toolbar
        tool_frame = tk.Frame(self, bg="white")
        tool_frame.pack(fill="x", padx=20, pady=10)

        tk.Label(tool_frame, text="Ημερομηνία:", bg="white",
font=("Arial", 10)).pack(side="left")
        self.cal = DateEntry(
            tool_frame, width=12, date_pattern="dd/mm/yyyy",
            background="#0A3D62", foreground="white",
showweeknumbers=False
        )
        self.cal.pack(side="left", padx=10)
        self.cal.bind("<<DateEntrySelected>>", lambda e:
self.display_available_slots())

        tk.Button(
            tool_frame, text="Προβολή Κλεισμένων",
command=self.load_booked_only,
            bg="#2C3E50", fg="white", relief="flat", padx=10
        ).pack(side="left", padx=5)

        tk.Button(
            tool_frame, text="Σήμερα",
command=lambda: [self.cal.set_date(datetime.date.today()),
self.display_available_slots()],
            bg="#2ecc71", fg="white", relief="flat", padx=10
        ).pack(side="left", padx=5)

        tk.Button(
            tool_frame, text="Διαγραφή",
command=self.delete_selected_appointment,
            bg="#EE5253", fg="white", relief="flat", padx=10
        ).pack(side="right", padx=5)

        tk.Button(

```

```

        tool_frame, text="Επεξεργασία",
command=self.edit_selected_appointment,
        bg="#F39C12", fg="white", relief="flat", padx=10
    ).pack(side="right", padx=5)

# Treeview
self.tree = ttk.Treeview(
    self, columns=("Act", "Time", "Status", "Employee",
"Details", "Notes"),
    show="headings", height=15
)
self.tree.heading("Act", text="+/-")
self.tree.heading("Time", text="Ωρα")
self.tree.heading("Status", text="Κατάσταση")
self.tree.heading("Employee", text="Υπάλληλος")
self.tree.heading("Details", text="Πελάτης", anchor="center" )
self.tree.heading("Notes", text="Σημειώσεις")
self.tree.column("Act", width=40, anchor="center")
self.tree.column("Time", width=150, anchor="center")
self.tree.column("Status", width=150, anchor="center")
self.tree.column("Employee", width=150, anchor="center")
self.tree.column("Details", width=400, anchor="center")
self.tree.column("Notes", width=200, anchor="center")
self.tree.pack(expand=True, fill="both", padx=20, pady=10)

self.tree.tag_configure("booked", foreground="#e74c3c")
self.tree.tag_configure("available", foreground="#27ae60")
self.tree.tag_configure("closed", foreground="#95a5a6",
font=("Arial", 10, "italic"))

self.tree.bind("<Button-1>", self.on_single_click)
self.tree.bind("<Double-1>", self.on_right_click)
self._appt_map = {} # Maps tree item to appt_id

self.display_available_slots()

def on_single_click(self, event):
    # Βρίσκω τη γραμμή που επιλεγθηκε από το χρήστη
    item = self.tree.identify_row(event.y)
    if not item:
        return

    # Ελέγχω αν η γραμμή αυτή έχει slot με ραντεβού
    if self.tree.get_children(item):
        # Παίρνω την τρέχουσα κατάσταση (αν είναι True=ανοιχτό ή
False=κλειστό)
        is_open = self.tree.item(item, "open")

```



```

        # Αλλάζω την κατάσταση στο αντίθετο (Toggle)
        self.tree.item(item, open=not is_open)

def alert_error(self, msg): messagebox.showerror("Σφάλμα", msg)
def alert_info(self, msg): messagebox.showinfo("Πληροφορία", msg)

def generate_dynamic_slots(self, date_obj):
    """Παράγει slots βάσει Business Settings."""
    settings = database.get_business_settings()
    try:
        sched = json.loads(settings.get("weekly_schedule", "{}"))
        duration = int(settings.get("slot_duration", 30))
    except:
        return []

    weekday_key =
["mon", "tue", "wed", "thu", "fri", "sat", "sun"][date_obj.weekday()]
    day_info = sched.get(weekday_key, {})

    if day_info.get("closed"):
        return []

    slots = []
    for s_prefix in ["s1", "s2"]:
        start_str = day_info.get(f"{s_prefix}_s")
        end_str = day_info.get(f"{s_prefix}_e")
        if start_str and end_str:
            try:
                curr = datetime.datetime.strptime(start_str,
"%H:%M")
                end = datetime.datetime.strptime(end_str, "%H:%M")
                while curr < end:
                    nxt = curr +
datetime.timedelta(minutes=duration)
                    slots.append(f"{curr.strftime('%H:%M')} -
{nxt.strftime('%H:%M')}")
                    curr = nxt
            except: pass
    return slots

def on_right_click(self, event):
    item = self.tree.identify_row(event.y)
    if not item: return

    self.tree.selection_set(item)
    val = self.tree.item(item, "values")

```

```

        # Αν η πρώτη στήλη (Act) έχει το + (ή όποιο σύμβολο βάλαμε για
        προσθήκη)
        if "+" in str(val[0]) or "[ + ]" in str(val[0]):
            clean_slot = str(val[1]) # Παίρνουμε την ώρα από τη 2η
            στήλη

            # Δυναμικό όριο βάσει ενεργών υπαλλήλων
            active_emp_count = len(database.get_active_employees())
            limit = max(1, active_emp_count)

            if len(self.tree.get_children(item)) >= limit:
                messagebox.showwarning("Πλήρες", f"Το slot είναι
                γεμάτο ({limit} ραντεβού)!")
                return

            win = CustomerSelector(self, None)
            win.callback = lambda cid, eid, notes:
            self.confirm_booking(clean_slot, cid, eid, notes, win)

            # Αν η πρώτη στήλη έχει το - (ή το [ - ] για διαγραφή)
            elif "-" in str(val[0]) or "[ - ]" in str(val[0]):
                self.delete_selected_appointment()

    def display_available_slots(self):
        # 1. Καθαρισμός πίνακα και δεδομένων
        for item in self.tree.get_children():
            self.tree.delete(item)
        self._appt_data = {}

        # 2. Λήψη επιλεγμένης ημερομηνίας
        sel_date = self.cal.get_date()
        date_str = self.cal.get()

        # 3. Έλεγχος Αργίας & Κυριακής
        is_h, h_name = is_holiday(sel_date)
        # sel_date.weekday() == 6 ελέγχει αν είναι Κυριακή
        if is_h or sel_date.weekday() == 6:
            reason = h_name if is_h else "Κυριακή"
            self.tree.insert("", "end", values=("-", "ΚΛΕΙΣΤΑ",
            f"Λόγω: {reason}"), tags=("closed",))
            return

        # 4. Παραγωγή των Slots βάσει ωραρίου
        slots = self.generate_dynamic_slots(sel_date)

```

```

        if not slots:
            self.tree.insert("", "end", values=("-", "ΚΛΕΙΣΤΑ", "Η
επιχείρηση δεν λειτουργεί"), tags=("closed",))
            return

        # 5. Λήψη των κλεισμένων ραντεβού (Ομαδοποίηση ανά ώρα)
        try:
            appts = database.get_day_appointments(date_str)
            # Φιλτράρουμε μόνο τα ενεργά (όχι διαγραμμένα)
            active_appts = [a for a in appts if "[DELETED]" not in
str(a.get('notes', ""))]
            # Δυναμικό όριο βάσει ενεργών υπαλλήλων
            active_emp_count = len(database.get_active_employees())
            limit = max(1, active_emp_count)
        except Exception as e:
            print(f"Σφάλμα βάσης: {e}")
            active_appts = []
            limit = 1

        # 6. Εμφάνιση των Slots με Ιεραρχία (Parent-Child)
        for s in slots:
            start_t = s.split(" - ")[0].strip()[:5]
            current_booked = [a for a in active_appts if
str(a['start_time'][:5]) == start_t]
            count = len(current_booked)

            # ΚΥΡΙΑ ΓΡΑΜΜΗ (PARENT) - Εδώ μπαίνει το [+]
            status_text = f"{count}/{limit} Κρατήσεις" if count > 0
            else "ΕΛΕΥΘΕΡΟ"
            parent_id = self.tree.insert("", "end",
                                         values=("+", s, status_text,
                                         "", "", "")),
                                         tags=("available",))

            # ΕΣΩΤΕΡΙΚΕΣ ΓΡΑΜΜΕΣ (CHILDREN) - Εδώ μπαίνει το [-]
            for a in current_booked:
                cust_name = a.get('customer_name') or "Άγνωστος"
                emp_name = a.get('employee_name') or "-"
                # Καθαρισμός σημειώσεων από tags για την εμφάνιση
                display_notes = str(a.get('notes',
                "")).replace("[MODIFIED]", "").replace("[DELETED]", "").strip()
                child_item = self.tree.insert(parent_id, "end",
                                              values=("-", "  L", "PANTEBOY", emp_name,
                                              cust_name, display_notes),
                                              tags=("booked",))
                self._appt_data[child_item] = a

```

```

def on_double_click(self, event):
    item = self.tree.identify_row(event.y)
    if not item: return
    val = self.tree.item(item, "values")

    # Αν πατήσει στην κύρια γραμμή (που έχει το [ + ])
    if "[ + ]" in str(val[0]):
        clean_slot = str(val[0]).replace("[ + ]", "").strip()

        # Δυναμικό όριο βάσει ενεργών υπαλλήλων
        active_emp_count = len(database.get_active_employees())
        limit = max(1, active_emp_count)

        # Έλεγχος αν έχουμε ήδη το όριο των ατόμων
        children = self.tree.get_children(item)
        if len(children) >= limit:
            return messagebox.showwarning("Πλήρες", f"Το slot έχει συμπληρώσει τα {limit} ραντεβού.")

        # 1. Δημιουργούμε το παράθυρο και το αποθηκεύουμε σε μια μεταβλητή (win)
        win = CustomerSelector(self, None)

        # 2. Περνάμε το 'win' μέσα στην confirm_booking για να ξέρει ποιο να κλείσει
        win.callback = lambda cid, eid, notes:
self.confirm_booking(
    clean_slot, cid, eid, notes, win
)

        # Αν πατήσει στο [ - ] μέσα στην ώρα
        elif "[ - ]" in str(val[0]):
            self.delete_selected_appointment()

def confirm_booking(self, slot_range, customer_id, employee_id, notes, popup_window):
    # 1. Καθαρισμός ώρας (Παίρνουμε το 09:00 από το 09:00 - 09:30 [ + ])
    start_time = slot_range.split(" - ")[0].strip()[:5]
    date_iso = self.cal.get_date().isoformat()

    # 2. Αποθήκευση στη Βάση
    success, msg = database.create_appointment(

```

```

        customer_id=customer_id,
        user_id=self.user["id"],
        employee_id=employee_id,
        date=self.cal.get(),
        time=start_time,
        notes=notes
    )

    # 3. Αποτέλεσμα & Ανανέωση
    if success:
        messagebox.showinfo("Επιτυχία", "Το ραντεβού
καταχωρήθηκε!")
        self.display_available_slots() # Εδώ θα φανεί το νέο
ραντεβού στο "βελάκι"
        popup_window.destroy() # Κλείνει το παράθυρο επιλογής
    else:
        messagebox.showerror("Σφάλμα", f"Αποτυχία: {msg}")

def delete_selected_appointment(self):
    # --- ΕΛΕΓΧΟΣ RBAC ---
    # Υποθέτουμε ότι το role_id = 1 είναι ο Admin.
    # Αν ο χρήστης δεν είναι Admin, σταματάμε τη διαδικασία.
    if self.role_id != 1:
        return self.alert_error("Δεν έχετε δικαιώματα
διαγραφής/ακύρωσης ραντεβού.")

    selected = self.tree.selection()
    if not selected:
        return self.alert_error("Παρακαλώ επιλέξτε ένα κλεισμένο
ραντεβού.")

    # Παίρνουμε τα δεδομένα του ραντεβού
    item = selected[0]
    appt = self._appt_data.get(item)

    if not appt:
        return self.alert_error("Μπορείτε να ακυρώσετε μόνο
υπάρχοντα ραντεβού.")

    if messagebox.askyesno("Επιβεβαίωση", f"Ακύρωση ραντεβού για:
{appt.get('customer_name')}"):
        # Αντί για οριστική διαγραφή, προσθέτουμε το tag [DELETED]
        current_notes = str(appt.get('notes', ""))
        if "[DELETED]" not in current_notes:
            new_notes = f"[DELETED] {current_notes}".strip()
        else:
            new_notes = current_notes

```

```

        # Παίρνουμε τη διάρκεια από το υπάρχον ραντεβού,
        current_duration = appt.get('duration', 30)

        # Ενημέρωση στη βάση (Soft Delete)
        success, err = database.update_appointment(
            appointment_id=appt['appointment_id'],
            customer_id=appt['customer_id'],
            user_id=appt.get('user_id', 1),
            date=appt['appt_date'],
            time=appt['start_time'],
            duration=current_duration,
            notes=new_notes, # Μαρκάρισμα για το Ιστορικό
            employee_id=appt.get('employee_id')
        )

        if success:
            self.alert_info("Το ραντεβού ακυρώθηκε επιτυχώς στο
ιστορικό.")
            self.display_available_slots() # Ανανέωση του
προγράμματος
        else:
            self.alert_error(f"Σφάλμα κατά την ακύρωση: {err}")

    def load_booked_only(self):
        for item in self.tree.get_children(): self.tree.delete(item)
        self._appt_data = {}
        date_str = self.cal.get()
        appts = database.get_day_appointments(date_str)

        for a in appts:
            t_str = f"{str(a['start_time'][:5])}"
            emp_name = a.get('employee_name') or "-"
            cust_name = a.get('customer_name') or "Άγνωστος"

            # Εισαγωγή και των 6 στηλών με τη σωστή σειρά
            item = self.tree.insert("", "end", values=(
                "-", t_str, "PANTEBOY", emp_name, cust_name,
a.get('notes', "")
            ), tags=("booked",))
            self._appt_data[item] = a

    def edit_selected_appointment(self):
        """Ανοίγει το νέο παράθυρο επεξεργασίας για το επιλεγμένο
ραντεβού."""
        sel = self.tree.selection()
        if not sel:

```

```

        return self.alert_error("Παρακαλώ επιλέξτε ένα κλεισμένο
ραντεβού από τη λίστα.")

    item = sel[0]
    appt = self._appt_data.get(item)

    if not appt:
        return self.alert_error("Μπορείτε να επεξεργαστείτε μόνο
υπάρχοντα ραντεβού (κόκκινα).")

    # Έλεγχος δικαιωμάτων (Admin ή Δημιουργός)
    if self.role_id != 1 and appt.get("user_id") !=
self.user["id"]:
        return messagebox.showwarning("Προσοχή", "Δεν έχετε
δικαίωμα να αλλάξετε ραντεβού άλλων χρηστών.")

    # ΑΝΟΙΓΜΑ ΤΟΥ ΝΕΟΥ ΠΑΡΑΘΥΡΟΥ (EditAppointmentWindow)
    # Του περνάμε το ραντεβού (appt) και τη συνάρτηση που θα κάνει
το save (confirm_edit)
    EditAppointmentWindow(self, appt, self.confirm_edit) # type:
ignore

    def confirm_edit(self, appt_id, customer_id, user_id, new_date,
new_time, duration, notes, new_emp_id):
        """Ενημέρωση ραντεβού χρησιμοποιώντας τα δεδομένα που
στάλθηκαν."""

        # Προσθήκη [MODIFIED] στα notes αν δεν υπάρχει
        clean_notes = str(notes)
        if "[MODIFIED]" not in clean_notes and "[DELETED]" not in
clean_notes:
            clean_notes = f"[MODIFIED] {clean_notes}".strip()

        # Κλήση της βάσης
        success, msg = database.update_appointment(
            appointment_id=appt_id,
            customer_id=customer_id,
            user_id=user_id,
            date=new_date,
            time=new_time,
            duration=duration,
            notes=clean_notes,
            employee_id=new_emp_id
        )

        if success:

```

```

        self.alert_info("Το ραντεβού ενημερώθηκε επιτυχώς!")
        self.display_available_slots()
    else:
        self.alert_error(f"Αποτυχία ενημέρωσης: {msg}")

class AppointmentSearch(tk.Frame):
    def __init__(self, parent):
        super().__init__(parent, bg="white")
        self._search_data = {} # Αποθηκεύει τα πλήρη δεδομένα του
        ραντεβού για κάθε γραμμή

        tk.Label(self, text="Αναζήτηση Ιστορικού Ραντεβού",
                  font=("Arial", 20, "bold"), bg="white",
                  fg="#0A3D62").pack(pady=20)

        # --- Toolbar για Αναζήτηση και Κουμπιά ---
        tool_frame = tk.Frame(self, bg="white")
        tool_frame.pack(fill="x", padx=50)

        tk.Label(tool_frame, text="Αναζήτηση:", bg="white",
                  font=("Arial", 11)).pack(side="left")
        self.search_entry = tk.Entry(tool_frame, font=("Arial", 12),
                                      relief="flat",
                                      highlightthickness=1,
                                      highlightbackground="#D9E2EC")
        self.search_entry.pack(side="left", padx=10, expand=True,
                                fill="x")
        self.search_entry.bind("<KeyRelease>", self.refresh_search)

        # Κουμπιά στα δεξιά
        tk.Button(tool_frame, text="Διαγραφή",
                  command=self.delete_selected,
                  bg="#EE5253", fg="white", relief="flat",
                  padx=15).pack(side="right", padx=5)

        tk.Button(tool_frame, text="Επεξεργασία",
                  command=self.edit_selected,
                  bg="#F39C12", fg="white", relief="flat",
                  padx=15).pack(side="right", padx=5)

        # --- Πίνακας Treeview ---
        self.tree = ttk.Treeview(self, columns=("Date", "Time",
        "Customer", "Phone", "Employee", "Notes", "Status"), show="headings",
        height=15)
        self.tree.heading("Date", text="Ημερομηνία")
        self.tree.heading("Time", text="Ωρα")
        self.tree.heading("Customer", text="Πελάτης")

```



```

self.tree.heading("Phone", text="Τηλέφωνο")
self.tree.heading("Employee", text="Υπάλληλος")
self.tree.heading("Status", text="Κατάσταση")
self.tree.heading("Notes", text="Σημειώσεις")
# Κεντράρισμα
for col in ("Date", "Time", "Customer", "Phone", "Employee",
"Status", "Notes"):
    self.tree.heading(col, anchor="center")
    self.tree.column(col, anchor="center")

self.tree.column("Date", width=120)
self.tree.column("Time", width=100)
self.tree.column("Customer", width=200)
self.tree.column("Phone", width=100)
self.tree.column("Employee", width=150)
self.tree.column("Status", width=120)
self.tree.column("Notes", width=150)
self.tree.pack(expand=True, fill="both", padx=50, pady=20)
self.tree.tag_configure("active_row", foreground="#27ae60") #
Πορτοκαλί
self.tree.tag_configure("edited_row", foreground="#F39C12")
self.tree.tag_configure("deleted_row", foreground="#e74c3c")

def refresh_search(self, event=None):
    query = self.search_entry.get().strip().lower()
    for item in self.tree.get_children():
        self.tree.delete(item)
    self._search_data = {}

    try:
        # 1. Φορτώνουμε Lookup για Πελάτες
        all_customers = database.get_all_customers()
        cust_lookup = {
            str(c.get('customer_id') or c.get('id')): {
                "name": f"{c.get('first_name', '')}
{c.get('last_name', '')}",
                "phone": str(c.get('phone', '-')),
                "email": str(c.get('email', '')).lower()
            } for c in all_customers
        }

        # 2. Παίρνουμε τα ραντεβού
        appts = get_all_appointments_historical()

        for a in appts:
            a_cust_id = str(a.get('customer_id') or "")

```

```

        cust_info = cust_lookup.get(a_cust_id, {"name":
"Άγνωστος", "phone": "-", "email": ""})

        cust_name = cust_info["name"]
        emp_name = a.get('employee_name') or "-"
        notes = str(a.get('notes', ""))

        # 3. Φίλτρο Αναζήτησης (Ψάχνει παντού: Πελάτη,
Υπάλληλο, Email)
        if query:
            clean_query = query.replace(" ", "")
            # Καθαρίζουμε και το τηλέφωνο του πελάτη από κενά
για τη σύγκριση
            clean_phone = cust_info["phone"].replace(" ", "")

            if not (query in cust_name.lower() or
                    query in emp_name.lower() or
                    query in cust_info["email"] or
                    clean_query in clean_phone):
                continue

        # 4. Format Ημερομηνίας
        raw_date = a.get('appt_date') or ""
        d_parts = raw_date.split("-")
        display_date =
f"{d_parts[2]}/{d_parts[1]}/{d_parts[0]}" if len(d_parts)==3 else
raw_date

        # 5. ΕΛΕΓΧΟΣ ΚΑΤΑΣΤΑΣΗΣ (Tags)
        if "[DELETED]" in notes:
            status_text = "ΔΙΑΓΡΑΜΜΕΝΟ"
            tag = "deleted_row"
        elif "[MODIFIED]" in notes:
            status_text = "ΤΡΟΠΟΠΟΙΗΜΕΝΟ"
            tag = "edited_row"
        else:
            status_text = "ΚΑΤΑΧΩΡΗΜΕΝΟ"
            tag = "active_row"

        display_notes = notes.replace("[MODIFIED]",
 "").replace("[DELETED]", "").strip()

        # 6. Εισαγωγή στον πίνακα
        item = self.tree.insert("", "end", values=(
            display_date,

```

```

        a.get('start_time') or "-",
        cust_name,
        cust_info["phone"],
        emp_name,
        display_notes,
        status_text
    ), tags=(tag,))

    self._search_data[item] = a

except Exception as e:
    messagebox.showerror("Σφάλμα Αναζήτησης", f"Παρουσιάστηκε
    πρόβλημα κατά την αναζήτηση:\n{e}")

def edit_selected(self):
    selected = self.tree.selection()
    if not selected:
        return messagebox.showwarning("Προσοχή", "Επιλέξτε ένα
        ραντεβού για επεξεργασία.")

    appt = self._search_data.get(selected[0])
    if appt:
        # Χρήση του EditAppointmentWindow που ήδη φτιάξαμε
        # Χρειαζόμαστε το 'parent' να έχει τη μέθοδο
        generate_dynamic_slots
        # Αν η κλάση αυτή δεν την έχει, μπορούμε να την πάρουμε
        από το Tab 1
        EditAppointmentWindow(self.master.wininfo_children()[0],
        appt, self.save_edit)

    def save_edit(self, appt_id, customer_id, user_id, new_date,
    new_time, duration, notes, new_emp_id):
        """Ενημέρωση ραντεβού από τη λίστα αναζήτησης/ιστορικού."""
        clean_notes = str(notes)
        if "[MODIFIED]" not in clean_notes and "[DELETED]" not in
        clean_notes:
            clean_notes = f"[MODIFIED] {clean_notes}".strip()

        success, error_msg = database.update_appointment(
            appointment_id=appt_id,
            customer_id=customer_id,
            user_id=user_id,
            date=new_date,
            time=new_time,
            duration=duration,
            notes=clean_notes,

```

```

        employee_id=new_emp_id
    )

    if success:
        messagebox.showinfo("Επιτυχία", "Το ραντεβού ενημερώθηκε  
επιτυχώς!")
        self.refresh_search()
    else:
        messagebox.showerror("Σφάλμα", f"Αποτυχία ενημέρωσης:  
{error_msg}")

    def delete_selected(self):
        selected = self.tree.selection()
        if not selected:
            return messagebox.showwarning("Προσοχή", "Επιλέξτε ένα  
ραντεβού για ακύρωση.")

        # Παίρνουμε τα δεδομένα από το dictionary
        item = selected[0]
        appt = self._search_data.get(item)
        if not appt: return

        if messagebox.askyesno("Επιβεβαίωση", f"Θέλετε να ακυρώσετε το  
ραντεβού του/της {appt.get('customer_name')};"):
            # Αντί για database.delete, κάνουμε UPDATE τις σημειώσεις  
με το tag [DELETED]
            old_notes = str(appt.get('notes', ""))
            if "[DELETED]" not in old_notes:
                new_notes = f"[DELETED] {old_notes}".strip()
            else:
                new_notes = old_notes

            # Ενημέρωση στη βάση των παιδιών
            success, err = database.update_appointment(
                appointment_id=appt['appointment_id'],
                customer_id=appt['customer_id'],
                user_id=appt.get('user_id', 1),
                date=appt['appt_date'],
                time=appt['start_time'],
                notes=new_notes, # Εδώ κλειδώνει η "διαγραφή" για το  
ιστορικό
                employee_id=appt.get('employee_id')
            )

            if success:
                messagebox.showinfo("Ολοκληρώθηκε", "Το ραντεβού  
σημειώθηκε ως ΔΙΑΓΡΑΜΜΕΝΟ στο ιστορικό.")

```

```

        self.refresh_search() # Φρεσκάρισμα για να γίνει
KOKKINO
        else:
            messagebox.showerror("Σφάλμα", err)

def get_all_appointments_historical():
    import sqlite3
    conn = database.get_connection()
    conn.row_factory = sqlite3.Row
    try:
        # ΑΛΛΑΓΗ: Από DESC το κάνουμε ASC για να ξεκινάει από το
        παλαιότερο
        query = """
            SELECT
                a.*,
                (c.first_name || ' ' || c.last_name) AS customer_name,
                (e.first_name || ' ' || e.last_name) AS employee_name
            FROM appointments a
            LEFT JOIN customers c ON a.customer_id = c.customer_id
            LEFT JOIN employees e ON a.employee_id = e.employee_id
            ORDER BY a.appt_date ASC, a.start_time ASC
        """
        cursor = conn.execute(query)

        rows = cursor.fetchall()
        results = [dict(r) for r in rows]
        return results
    except Exception as e:
        print(f"Σφάλμα SQL: {e}")
        return []
    finally:
        conn.close()

```

Stats.py

```

"""
=====
=====
APXΕΙΟ: gui_stats.py
ΣΚΟΠΟΣ: Εμφάνιση στατιστικών γραφημάτων (matplotlib ενσωματωμένο σε
tkinter)
=====
=====

```

```

"""
import tkinter as tk
from tkinter import ttk, messagebox
from typing import Any, List, Dict, Tuple, Callable, Optional
import datetime
import calendar
import json
from tkcalendar import DateEntry

from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

import database
import export_data

# --- GUI CONSTANTS (Aegean Theme) ---
COLOR_BG      = "#F7F9FC"
COLOR_SIDEBAR = "#0A3D62"
COLOR_ACCENT  = "#1E90FF"
COLOR_WHITE   = "#FFFFFF"
COLOR_TEXT    = "#1B1F23"
COLOR_BORDER  = "#E5E9F0"
COLOR_GREEN   = "#167C52"
COLOR_ORANGE  = "#F39C12"
COLOR_GRAY    = "#636E72"
FONT_TITLE    = ("Arial", 18, "bold")
# -----

class StatsPanel(tk.Frame):
    """
    Κλάση που δημιουργεί ένα Frame με στατιστικά γραφήματα.
    Ενσωματώνεται στο κεντρικό content_frame του MainWindow.
    """

    def __init__(self, parent: tk.Widget, user_profile:
Optional[Dict[str, Any]] = None) -> None:
        super().__init__(parent)
        self.configure(bg=COLOR_BG) # Συγχρονισμός με το background
του dashboard
        self.pack(fill="both", expand=True) # Απαραίτητο: τοποθετεί
το Frame στο content_frame

        # =====
        # HEADER
        # =====
        header_frame = tk.Frame(self, bg=COLOR_BG)

```

```

header_frame.pack(side="top", fill="x", padx=20, pady=(20,
10))

tk.Label(
    header_frame,
    text="Στατιστικά & Αναφορές",
    font=("Arial", 18, "bold"),
    bg=COLOR_BG,
    fg=COLOR_SIDE BAR,
).pack(side="left")

# =====
# TOP BUTTONS (NAV)
# =====
nav_frame = tk.Frame(self, bg=COLOR_BG)
nav_frame.pack(side="top", fill="x", padx=20, pady=10)

style = ttk.Style()
style.configure("Stats.TButton", font=("Arial", 10))

self.btn_daily = ttk.Button(
    nav_frame, text="Ανά Ημέρα", command=self._show_daily,
style="Stats.TButton"
)
self.btn_daily.pack(side="left", padx=5)

self.btn_monthly = ttk.Button(
    nav_frame, text="Ανά Μήνα", command=self._show_monthly,
style="Stats.TButton"
)
self.btn_monthly.pack(side="left", padx=5)

self.btn_customer = ttk.Button(
    nav_frame, text="Top 10 Πελάτες",
command=self._show_by_customer, style="Stats.TButton"
)
self.btn_customer.pack(side="left", padx=5)

self.btn_employee = ttk.Button(
    nav_frame, text="Απόδοση Υπαλλήλων",
command=self._show_by_employee, style="Stats.TButton"
)
self.btn_employee.pack(side="left", padx=5)

self.btn_export = ttk.Button(

```

```

        nav_frame, text="Εξαγωγή", command=self._show_export_menu,
style="Stats.TButton"
    )
    self.btn_export.pack(side="right", padx=5)

    # =====
    # CHART CONTAINER
    # =====
    # Χρησιμοποιούμε ένα λευκό frame για το γράφημα ώστε να
    μοιάζει με κάρτα
    self.container = tk.Frame(
        self,
        bg=COLOR_WHITE,
        highlightthickness=1,
        highlightbackground=COLOR_BORDER,
        padx=10,
        pady=10,
    )
    self.container.pack(fill="both", expand=True, padx=20,
pady=20)

    # Frame για τα διαδραστικά φίλτρα/widgets
    self.controls_frame = tk.Frame(self.container, bg=COLOR_WHITE)
    self.controls_frame.pack(side="top", fill="x", pady=(0, 10))

    self.canvas = None
    self.current_data = []
    self.current_report_type = ""
    self.current_date_str = ""
    self.current_month_str = ""

    # Προεπιλεγμένο γράφημα κατά την εκκίνηση
    self.after(100, self._show_daily)

    def _clear_controls(self) -> None:
        # Καθαρισμός των widgets φίλτρων από το controls_frame
        for widget in self.controls_frame.winfo_children():
            widget.destroy()

    def _range_times_local(self, start_str: str, end_str: str,
step_min: int) -> List[str]:
        # Παραγωγή λίστας με slots ωραρίων
        time_slots_list = []
        try:
            start = datetime.datetime.strptime(start_str, "%H:%M")
            end = datetime.datetime.strptime(end_str, "%H:%M")
            curr = start

```



```

        while curr < end:
            time_slots_list.append(curr.strftime("%H:%M"))
            curr += datetime.timedelta(minutes=step_min)
    except Exception:
        pass
    return time_slots_list

def _draw_chart(self, fig: Figure) -> None:
    """Καθαρίζει το προηγούμενο γράφημα και σχεδιάζει το νέο στο
    canvas."""
    if self.canvas is not None:
        self.canvas.get_tk_widget().destroy()

    self.canvas = FigureCanvasTkAgg(fig, master=self.container)
    self.canvas.draw()
    self.canvas.get_tk_widget().pack(fill="both", expand=True)

def _show_daily(self) -> None:
    # Καθαρισμός φίλτρων και δημιουργία DateEntry
    self._clear_controls()

    lbl_date = tk.Label(
        self.controls_frame,
        text="Επιλογή Ημερομηνίας:",
        bg=COLOR_WHITE,
        fg=COLOR_TEXT,
        font=("Arial", 10, "bold")
    )
    lbl_date.pack(side="left", padx=(10, 5))

    self.date_picker = DateEntry(
        self.controls_frame,
        width=12,
        date_pattern="dd/mm/yyyy",
        font=("Arial", 10)
    )
    self.date_picker.pack(side="left", padx=5)
    self.date_picker.bind("<<DateEntrySelected>>", lambda e:
self._update_daily_chart())

    # Αρχική εμφάνιση γραφήματος
    self._update_daily_chart()

def _update_daily_chart(self) -> None:
    # Λήψη επιλεγμένης ημερομηνίας
    selected_date = self.date_picker.get_date()
    date_str_dmy = selected_date.strftime("%d/%m/%Y")

```

```

self.current_date_str = date_str_dmy

# Ανάκτηση ρυθμίσεων ωραρίου
try:
    settings = database.get_business_settings()
    weekly_schedule =
json.loads(settings.get("weekly_schedule", "{}"))
    slot_duration = int(settings.get("slot_duration", 30))
except Exception:
    weekly_schedule = {}
    slot_duration = 30

# Εύρεση ημέρας της εβδομάδας
weekday_key = ["mon", "tue", "wed", "thu", "fri", "sat",
"sun"][selected_date.weekday()]
day_info = weekly_schedule.get(weekday_key, {})

# Έλεγχος αν η επιχείρηση είναι κλειστή
if day_info.get("closed") or (not day_info.get("s1_s") and not
day_info.get("s2_s")):
    self._show_message(f"Η επιχείρηση είναι κλειστή την ημέρα
αυτή ({date_str_dmy})")
    self._save_current_data([], "daily")
    return

# Παραγωγή slots
slots = []
s1_s, s1_e = day_info.get("s1_s"), day_info.get("s1_e")
if s1_s and s1_e:
    slots.extend(self._range_times_local(s1_s, s1_e,
slot_duration))
s2_s, s2_e = day_info.get("s2_s"), day_info.get("s2_e")
if s2_s and s2_e:
    slots.extend(self._range_times_local(s2_s, s2_e,
slot_duration))

if not slots:
    self._show_message(f"Δεν υπάρχουν ορισμένα ωράρια για αυτή
την ημέρα ({date_str_dmy})")
    self._save_current_data([], "daily")
    return

# Ανάκτηση ραντεβού
appointments = database.get_day_appointments(date_str_dmy)

# Καταμέτρηση ανά slot
counts = []

```

```

        for slot in slots:
            count = sum(1 for appt in appointments if
appt.get("start_time") == slot)
            counts.append(count)

        # Σχεδίαση Bar Chart
        fig = Figure(figsize=(8, 5), dpi=100, facecolor="white")
        ax = fig.add_subplot(111)

        ax.bar(slots, counts, color="#1E90FF", edgecolor="#0A3D62",
alpha=0.7)
        ax.set_title(f"Ωριαία Κατανομή Παντεβού – {date_str_dmy}",
fontsize=12, pad=15)
        ax.set_ylabel("Πλήθος Παντεβού")
        ax.grid(axis="y", linestyle="--", alpha=0.3)

        # Απαίωση ετικετών X (ανά 2 slots)
        indices = list(range(0, len(slots), 2))
        if (len(slots) - 1) not in indices:
            indices.append(len(slots) - 1)
        ax.set_xticks(indices)
        ax.set_xticklabels([slots[i] for i in indices], rotation=30,
ha="right")

        fig.tight_layout()
        self._draw_chart(fig)

        # Αποθήκευση για εξαγωγή
        export_records = [{"date": slot, "count": count} for slot,
count in zip(slots, counts)]
        self._save_current_data(export_records, "daily")

    def _show_monthly(self) -> None:
        # Καθαρισμός φίλτρων και δημιουργία Combobox
        self._clear_controls()

        lbl_month = tk.Label(
            self.controls_frame,
            text="Επιλογή Μήνα:",
            bg=COLOR_WHITE,
            fg=COLOR_TEXT,
            font=("Arial", 10, "bold")
        )
        lbl_month.pack(side="left", padx=(10, 5))

        # Ανάκτηση μηνών με κρατήσεις

```

```

chart_records = database.get_appointment_count_by_month()
months_list = [row["month"] for row in chart_records]

# Fallback στον τρέχοντα μήνα
current_month_str = datetime.date.today().strftime("%m/%Y")
if current_month_str not in months_list:
    months_list.append(current_month_str)

self.month_picker = ttk.Combobox(
    self.controls_frame,
    values=months_list,
    state="readonly",
    font=("Arial", 10)
)
self.month_picker.pack(side="left", padx=5)

if current_month_str in months_list:
    self.month_picker.set(current_month_str)
else:
    self.month_picker.current(len(months_list) - 1)

self.month_picker.bind("<<ComboboxSelected>>", lambda e:
self._update_monthly_chart())

# Αρχική εμφάνιση γραφήματος
self._update_monthly_chart()

def _update_monthly_chart(self) -> None:
    # Λήψη επιλεγμένου μήνα
    selected_month = self.month_picker.get()
    self.current_month_str = selected_month

    month_str, year_str = selected_month.split("/")
    m = int(month_str)
    y = int(year_str)

    # Δημιουργία χρονολογίου για όλο το μήνα
    _, num_days = calendar.monthrange(y, m)
    labels = []
    values = []

    # Λήψη δεδομένων
    chart_records = database.get_appointment_count_by_date()
    records_dict = {row["date"]: row["count"] for row in
chart_records}

```

```

        for day in range(1, num_days + 1):
            iso_date = f"{y:04d}-{m:02d}-{day:02d}"
            labels.append(f"{day:02d}/{m:02d}")
            values.append(records_dict.get(iso_date, 0))

        # Σχεδίαση Wave/Area Chart
        fig = Figure(figsize=(8, 5), dpi=100, facecolor="white")
        ax = fig.add_subplot(111)

        ax.plot(labels, values, color="#1E90FF", linewidth=2)
        ax.fill_between(labels, values, color="#1E90FF", alpha=0.15)

        ax.set_title(f"Τάση Παντεβού - {selected_month}", fontsize=12,
pad=15)
        ax.set_ylabel("Πλήθος Παντεβού")
        ax.grid(True, linestyle=":", alpha=0.6)

        # Αραίωση ετικετών X (ανά 5 ημέρες)
        indices = list(range(0, len(labels), 5))
        if (len(labels) - 1) not in indices:
            indices.append(len(labels) - 1)
        ax.set_xticks(indices)
        ax.set_xticklabels([labels[i] for i in indices], rotation=30,
ha="right")

        fig.tight_layout()
        self._draw_chart(fig)

        # Αποθήκευση για εξαγωγή
        export_records = [{"day": label, "count": count} for label,
count in zip(labels, values)]
        self._save_current_data(export_records, "monthly")

    def _show_by_customer(self) -> None:
        self._clear_controls()
        # Εμφανίζει οριζόντιο bar chart με τους Top 10 πελάτες με βάση
το πλήθος των ραντεβού τους.
        chart_records = database.get_appointment_count_by_customer()

        self._save_current_data(chart_records, "customer")
        # Τα δεδομένα έρχονται ταξινομημένα φθίνουσα, τα αντιστρέφουμε
για το barh
        labels = [row["name"] for row in chart_records[:10]][::-1]
        values = [row["count"] for row in chart_records[:10]][::-1]

        fig = Figure(figsize=(8, 5), dpi=100, facecolor="white")

```

```

        ax = fig.add_subplot(111)

        ax.barh(labels, values, color="#F39C12", alpha=0.8)
        ax.set_title("Top 10 Πελάτες (Συνολικά Ραντεβού)",
        fontsize=12, pad=15)
        ax.set_xlabel("Αριθμός Ραντεβού")

        fig.tight_layout()
        self._draw_chart(fig)

    def _show_by_employee(self) -> None:
        self._clear_controls()
        # Εμφανίζει οριζόντιο bar chart με την απόδοση των υπαλλήλων
        βάσει των συνολικών ραντεβού τους.
        chart_records = database.get_appointment_count_by_employee()

        self._save_current_data(chart_records, "employee")
        # Τα δεδομένα έρχονται ταξινομημένα φθίνουσα, τα αντιστρέφουμε
        για το barh
        labels = [row["name"] for row in chart_records][::-1]
        values = [row["count"] for row in chart_records][::-1]

        fig = Figure(figsize=(8, 5), dpi=100, facecolor="white")
        ax = fig.add_subplot(111)

        ax.barh(labels, values, color="#1E90FF", alpha=0.8)
        ax.set_title("Απόδοση Υπαλλήλων (Συνολικά Ραντεβού)",
        fontsize=12, pad=15)
        ax.set_xlabel("Αριθμός Ραντεβού")

        fig.tight_layout()
        self._draw_chart(fig)

    def _show_message(self, message: str) -> None:
        """Εμφανίζει ένα μήνυμα κειμένου αντί για γράφημα."""
        fig = Figure(figsize=(8, 5), dpi=100, facecolor="white")
        ax = fig.add_subplot(111)
        ax.text(0.5, 0.5, message, ha="center", va="center",
        fontsize=12, color="#636E72")
        ax.set_axis_off()
        self._draw_chart(fig)

    # =====
    # EXPORT LOGIC
    # =====
    def _show_export_menu(self) -> None:

```

```

        """Εμφανίζει μενού επιλογών για την εξαγωγή των
        στατιστικών."""
        menu = tk.Menu(self, tearoff=0)
        menu.add_command(label="Εξαγωγή σε Excel (.xlsx)",
            command=lambda: self._perform_export("xlsx"))
        menu.add_command(label="Εξαγωγή σε CSV (.csv)",
            command=lambda: self._perform_export("csv"))
        menu.add_command(label="Εξαγωγή σε PDF (.pdf)",
            command=lambda: self._perform_export("pdf"))
        menu.post(self.wininfo_pointerx(), self.wininfo_pointery())

    def _perform_export(self, export_format: str) -> None:
        """Εξάγει τα τρέχοντα στατιστικά δεδομένα."""
        if not self.current_data:
            messagebox.showwarning("Προσοχή", "Δεν υπάρχουν δεδομένα
            για εξαγωγή.")
            return

        # Προετοιμασία δεδομένων για την export_data.py
        export_list = []

        if self.current_report_type == "daily":
            headers = {"col1": "Ώρα", "col2": "Πλήθος Ραντεβού"}
            title = f"Ημερήσια Κατανομή Ραντεβού -
            {self.current_date_str}"
            filename =
            f"stats_daily_{self.current_date_str.replace('/', '_')}"
            elif self.current_report_type == "monthly":
                headers = {"col1": "Ημερομηνία", "col2": "Πλήθος
                Ραντεβού"}
                title = f"Στατιστικά Ραντεβού - {self.current_month_str}"
                filename =
                f"stats_monthly_{self.current_month_str.replace('/', '_')}"
            elif self.current_report_type == "customer":
                headers = {"col1": "Πελάτης", "col2": "Συνολικά Ραντεβού"}
                title = "Στατιστικά: Top 10 Πελάτες"
                filename = "stats_top_customers"
            else: # employee
                headers = {"col1": "Υπάλληλος", "col2": "Συνολικά
                Ραντεβού"}
                title = "Στατιστικά: Απόδοση Υπαλλήλων"
                filename = "stats_employee_performance"

        for data_row in self.current_data:
            if self.current_report_type == "daily":
                export_list.append({"col1": data_row["date"], "col2":
                data_row["count"]})

```

```
        elif self.current_report_type == "monthly":
            export_list.append({"col1": data_row["day"], "col2":
data_row["count"]})
        else: # customer or employee
            export_list.append({"col1": data_row["name"], "col2":
data_row["count"]})

    if export_format == "csv":
        export_data.export_to_csv(export_list, headers, filename)
    elif export_format == "xlsx":
        export_data.export_to_xlsx(export_list, headers, filename,
"Στατιστικά")
    elif export_format == "pdf":
        export_data.export_to_pdf(export_list, headers, filename,
title)

    def _save_current_data(self, chart_records: List[Dict[str, Any]],
report_type: str) -> None:
        self.current_data = chart_records
        self.current_report_type = report_type
```


Πηγές

VIDEOS

[1] Tkinter Course - Create Graphic User Interfaces in Python Tutorial:

<https://www.youtube.com/watch?v=YXPyB4XeYLA>

[2] Learn Tkinter in 20 minutes:

<https://www.youtube.com/watch?v=ISNlrR1nZU>

[3] Tkinter Beginner Course - Python GUI Development

<https://www.youtube.com/watch?v=ibf5cx221hk>

[4] Python Tkinter Tutorial for Beginners

https://www.youtube.com/playlist?list=PLZPZq0r_RZOOeQBaP5SeMjl2nwDcJaV0T

e – Books / Σημειώσεις

[1] Παραδείγματα GUI με την Tkinter: <https://liagason.com/simlpe-gui-in-python>

[2] Python Tkinter Παραθυρικές εφαρμογές με Python, διαθέσιμα [εδώ](#)

[3] Python – Εισαγωγή στο Tkinter, διαθέσιμα [εδώ](#)

[4] Γραφική διεπαφή χρήστη με Python, διαθέσιμα [εδώ](#)

[5] Μαθαίνω Python και Tkinter, e – book, διαθέσιμο [εδώ](#)

[6] Python GUI – tkinter, διαθέσιμο από το: <https://www.geeksforgeeks.org/python-gui-tkinter/>

[7] Official Tkinter Python Library Tutorial, available at:

<https://docs.python.org/3/library/tk.html>

[8] Tkinter 8.5 reference: aGUI for Python: <https://tkdocs.com/shipman/tkinter.pdf>

[9] Learning Tkinter: <https://riptutorial.com/Download/tkinter.pdf>

[10] Tkinter GUI Programming by Example:

<https://edu.anarcho-copy.org/Programming%20Languages/Python/tkinter-gui-programming-example.pdf>

[11] <https://www.tutorialspoint.com/python/index.htm>

[12] <https://www.pythontutorial.net/tkinter/>

[13] <https://realpython.com/python-gui-tkinter/>